

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC PHENIKAA



BÁO CÁO ĐỒ ÁN LIÊN NGÀNH

**Đề tài: Lập trình ứng dụng PU Connection – Phát triển hệ thống
Phenikaa University connection trên di động dưới dạng mạng xã hội cho
sinh viên về việc chia sẻ thông tin, trao đổi học tập**

Sinh viên thực hiện:	Đặng Quang Vi – 21010583 Phạm Quang Hùng - 21011605
Học phần:	CSE703014
Ngành học:	Công nghệ thông tin
Giảng viên hướng dẫn:	ThS Vũ Quang Dũng

HÀ NỘI, 2024

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC PHENIKAA



BÁO CÁO ĐỒ ÁN LIÊN NGÀNH

**Đề tài: Lập trình ứng dụng PU Connection – Phát triển hệ thống
Phenikaa University connection trên di động dưới dạng mạng xã hội cho
sinh viên về việc chia sẻ thông tin, trao đổi học tập**

Sinh viên thực hiện:	Đặng Quang Vi – 21010583 Phạm Quang Hùng - 21011605
Học phần:	CSE703014
Ngành học:	Công nghệ thông tin
Giảng viên hướng dẫn:	ThS Vũ Quang Dũng

HÀ NỘI, 2024

MỤC LỤC

1.	<i>Yêu cầu</i>	6
1.1.	Đặt vấn đề.....	6
1.1.1.	Mô tả vấn đề	6
1.1.2.	Giải pháp	6
1.2.	Công nghệ sử dụng.....	8
	Flutter.....	8
	Appwrite	8
	MariaDB	8
	Dart	8
	RESTful API.....	8
2.	<i>Phân tích thiết kế hệ thống</i>	9
2.1.	Mô hình kiến trúc hệ thống	9
2.2.	Use case.....	11
2.2.1.	Mô tả chi tiết use-case Đăng ký	12
2.2.2.	Mô tả chi tiết use-case Đăng nhập	14
2.2.3.	Mô tả chi tiết use-case đăng status	16
2.2.4.	Mô tả chi tiết use-case xem status.....	18
2.2.5.	Mô tả chi tiết use-case Tương tác bài viết (Like status)	19
2.2.6.	Mô tả chi tiết use-case Bình luận	22
2.2.7.	Mô tả chi tiết use-case share status	24
2.2.8.	Mô tả chi tiết use-case Tìm kiếm status	26
2.2.9.	Mô tả chi tiết use-case Thông báo.....	28
2.2.10.	Mô tả chi tiết use-case Quản lý profile	29
2.3.	Các biểu đồ tuần tự	32
2.3.1.	Biểu đồ tuần tự Đăng nhập.....	32
2.3.2.	Biểu đồ tuần tự Đăng ký.....	33
2.3.3.	Biểu đồ tuần tự Đăng status	34
2.3.4.	Biểu đồ tuần tự Tương tác bài viết (Like status).....	35
2.3.5.	Biểu đồ tuần tự comment status	36
2.3.6.	Biểu đồ tuần tự share status.....	37

2.3.7.	Biểu đồ tuần tự Tìm kiếm.....	38
2.3.8.	Biểu đồ tuần tự Thông báo	39
2.3.9.	Biểu đồ tuần tự Chỉnh sửa thông tin cá nhân	40
2.4.	Mô hình thiết kế lớp.....	41
2.5.	Thiết kế cơ sở dữ liệu.....	45
3.	<i>Thực nghiệm và kết quả</i>	48
3.1.	Mô hình tổng quan	48
3.2.	Các chức năng chính	49
4.	<i>Kết luận và hướng phát triển</i>	53
4.1.	Kết luận	53
4.2.	Hướng phát triển sản phẩm	54
4.3.	Đóng góp của thành viên nhóm	55
	<i>TÀI LIỆU THAM KHẢO</i>	56

DANH MỤC HÌNH ẢNH

Hình 1:	Mô hình hệ thống tổng quan	9
Hình 2:	Mô hình use-case Tổng quát	11
Hình 3:	Mô hình use-case đăng ký.....	12
Hình 4:	Activity Diagram cho Đăng ký.....	12
Hình 5:	Mô hình use case đăng nhập	14
Hình 6:	Activity Diagram cho Đăng nhập.....	14
Hình 7	Use-case đăng status.....	16
Hình 8:	Activity Diagram cho đăng status	16
Hình 9.	Use-case xem status.....	18
Hình 10:	Activity Diagram cho xem status	18
Hình 11:	Mô hình use-case like status.....	20

Hình 12: Activity Diagram cho Tương tác bài viết (Thả tim).....	20
Hình 13: Mô hình use-case bình luận.....	22
Hình 14: Activity Diagram cho Bình luận.....	22
Hình 15: Mô hình use-case share status	24
Hình 16: Activity Diagram cho Share status	24
Hình 17: Mô hình use-case tìm kiếm status	26
Hình 18: Activity Diagram cho Tìm kiếm.....	26
Hình 19: Activity Diagram cho Thông báo	28
Hình 20: Mô hình use-case quản lý profile	29
Hình 21: Activity Diagram cho Chỉnh sửa profile	30
Hình 22: Biểu đồ tuần tự Đăng nhập.....	32
Hình 23: Biểu đồ tuần tự Đăng kí	33
Hình 24: Biểu đồ tuần tự Đăng status	34
Hình 25: Biểu đồ tuần tự Tương tác bài viết (Like status).....	35
Hình 26: Biểu đồ tuần tự comment status	36
Hình 27: Biểu đồ tuần tự share status.....	37
Hình 28: Biểu đồ tuần tự Tìm kiếm.....	38
Hình 29: Biểu đồ tuần tự Thông báo	39
Hình 30: Biểu đồ tuần tự Chỉnh sửa thông tin cá nhân	40
Hình 31: Sơ đồ thiết kế lớp	41
Hình 32: Mô hình thiết kế cơ sở dữ liệu.....	45
Hình 33: Mô hình hệ thống tổng quan	48
Hình 34: Đăng nhập bằng tài khoản Phenikaa và xem lịch học trên QLDT.....	49
Hình 35: Tìm kiếm user.....	50
Hình 36: Giao diện bài viết	51
Hình 37: Giao diện messenger	52

1. Yêu cầu

1.1. Đặt vấn đề

1.1.1. Mô tả vấn đề

Trong thời đại số hóa ngày nay, nhu cầu kết nối và chia sẻ thông tin giữa các cá nhân và tổ chức ngày càng trở nên quan trọng. Các mạng xã hội và nền tảng trực tuyến đã trở thành công cụ không thể thiếu, giúp người dùng giao tiếp, chia sẻ kinh nghiệm và duy trì mối quan hệ. Đáp ứng nhu cầu này, ứng dụng PU Connection được phát triển nhằm tạo ra một nền tảng kết nối hiệu quả và tiện lợi. Ứng dụng cung cấp các chức năng cơ bản nhưng cần thiết như đăng ký, đăng nhập, tìm kiếm, đăng trạng thái, xem hồ sơ và quên mật khẩu. Mục tiêu của PU Connection là cung cấp một giao diện thân thiện, dễ sử dụng, đồng thời đảm bảo bảo mật và sự riêng tư cho người dùng. Ứng dụng giúp người dùng dễ dàng kết nối, chia sẻ thông tin và tìm kiếm dữ liệu cá nhân một cách nhanh chóng, hiệu quả. Trong bối cảnh sự phát triển không ngừng của công nghệ và gia tăng nhu cầu kết nối trực tuyến, việc phát triển PU Connection không chỉ đáp ứng nhu cầu kết nối và chia sẻ thông tin, mà còn đóng góp vào việc xây dựng một cộng đồng trực tuyến gắn kết và mạnh mẽ.

1.1.2. Giải pháp

- Sử dụng Flutter để phát triển ứng dụng di động đa nền tảng: Flutter là một bộ công cụ UI của Google cho phép phát triển ứng dụng di động nhanh chóng và dễ dàng trên cả iOS và Android từ một mã nguồn duy nhất. Bằng cách sử dụng Flutter, chúng ta có thể:

- Tăng tốc quá trình phát triển và giảm chi phí bằng cách sử dụng cùng một cơ sở mã cho cả hai nền tảng.
- Cung cấp giao diện người dùng nhất quán và hiệu năng cao trên cả iOS và Android.
- Dễ dàng bảo trì và cập nhật ứng dụng với các thay đổi đồng bộ trên tất cả các nền tảng.

- Thiết kế cơ sở dữ liệu bảo mật và hiệu quả

Để đảm bảo tính bảo mật và hiệu quả trong việc lưu trữ và truy xuất dữ liệu người dùng, chúng tôi đề xuất:

- Sử dụng cơ sở dữ liệu Firebase, một giải pháp lưu trữ đám mây của Google, với các tính năng bảo mật mạnh mẽ và khả năng mở rộng tốt.

- Triển khai các biện pháp bảo mật như mã hóa dữ liệu, xác thực hai yếu tố (2FA), và kiểm soát truy cập chặt chẽ.
 - Tối ưu hóa cơ sở dữ liệu để đảm bảo truy vấn nhanh chóng và hiệu quả, hỗ trợ tốt cho các chức năng tìm kiếm và truy xuất dữ liệu.
- Đảm bảo trải nghiệm người dùng (UX) tốt
- Để cung cấp trải nghiệm người dùng tốt nhất, chúng tôi sẽ:
- Thiết kế giao diện người dùng (UI) trực quan, thân thiện và dễ sử dụng.
 - Tối ưu hóa hiệu năng ứng dụng để đảm bảo thời gian phản hồi nhanh và mượt mà.
 - Thu thập phản hồi từ người dùng để liên tục cải tiến và cập nhật các tính năng của ứng dụng.
- Triển khai và duy trì ứng dụng
- Chúng tôi sẽ triển khai ứng dụng PU Connection trên các cửa hàng ứng dụng (App Store và Google Play), và thực hiện các bước duy trì bao gồm:
- Cập nhật định kỳ để cải thiện tính năng và vá lỗi.
 - Theo dõi và phân tích hiệu suất ứng dụng để tối ưu hóa và cải tiến.
 - Hỗ trợ kỹ thuật và giải đáp thắc mắc cho người dùng.
- Bằng cách thực hiện các giải pháp trên, chúng tôi tin rằng ứng dụng PU Connection sẽ đáp ứng tốt nhu cầu kết nối và chia sẻ thông tin của người dùng, đồng thời đảm bảo tính bảo mật và sự tiện lợi trong quá trình sử dụng.

1.2. Công nghệ sử dụng

Flutter

- **Flutter** là một framework phát triển ứng dụng di động mã nguồn mở do Google phát triển. Nó cho phép xây dựng các ứng dụng di động đẹp mắt và hiệu năng cao cho cả Android và iOS từ cùng một codebase. Flutter sử dụng ngôn ngữ lập trình Dart và cung cấp một bộ công cụ giao diện người dùng phong phú và linh hoạt.

Appwrite

- **Appwrite** là một backend mã nguồn mở được thiết kế để giúp các nhà phát triển xây dựng các ứng dụng hiện đại một cách dễ dàng và nhanh chóng. Appwrite cung cấp một loạt các API và dịch vụ như cơ sở dữ liệu, xác thực người dùng, lưu trữ files, và các dịch vụ khác, giúp giảm thiểu công việc backend và tập trung vào phát triển frontend.

MariaDB

- **MariaDB** là một hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, tương thích với MySQL và được sử dụng trong Appwrite để lưu trữ dữ liệu ứng dụng. MariaDB cung cấp hiệu năng cao, tính năng bảo mật tốt, và khả năng mở rộng linh hoạt, phù hợp cho các ứng dụng yêu cầu tính nhất quán và toàn vẹn dữ liệu.

Dart

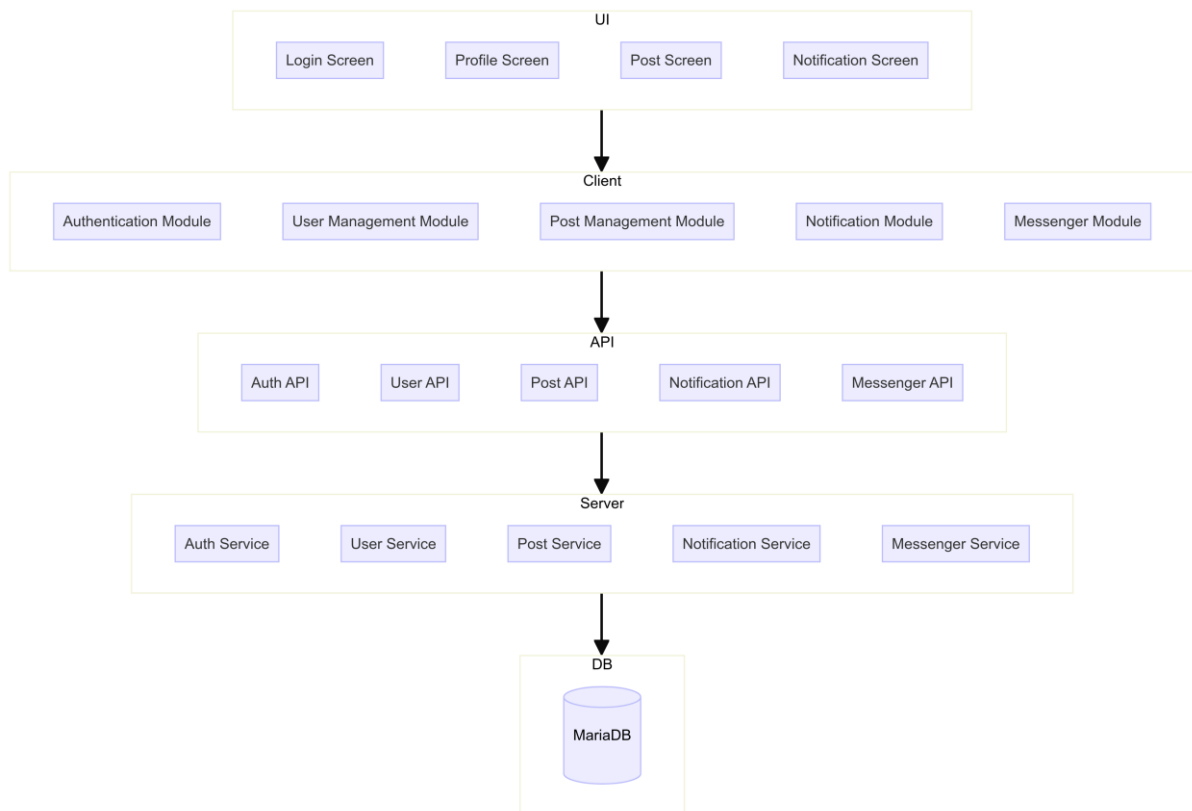
- **Dart** là ngôn ngữ lập trình chính được sử dụng trong Flutter. Dart được thiết kế để phát triển ứng dụng frontend với hiệu năng cao, cú pháp đơn giản và dễ học, đặc biệt thích hợp cho việc xây dựng giao diện người dùng.

RESTful API

- **RESTful API** là một phong cách kiến trúc cho các dịch vụ web, cho phép các hệ thống giao tiếp với nhau qua HTTP. Trong ứng dụng PU Connection, RESTful API được sử dụng để giao tiếp giữa ứng dụng Flutter và backend Appwrite, cho phép trao đổi dữ liệu và thực hiện các thao tác CRUD (Create, Read, Update, Delete).

2. Phân tích thiết kế hệ thống

2.1. Mô hình kiến trúc hệ thống



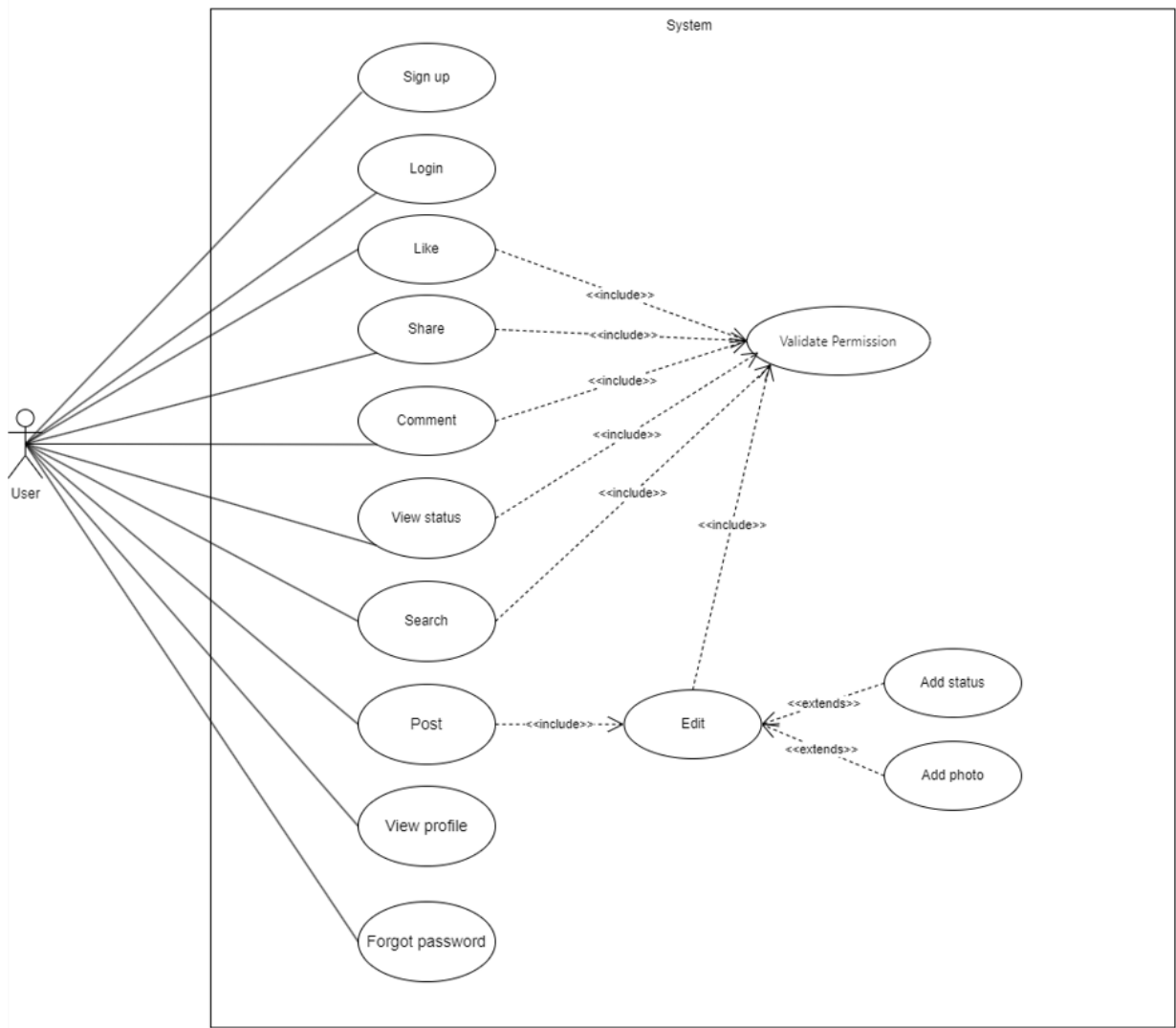
Hình 1: Mô hình hệ thống tổng quan

Mô tả kiến trúc

- **UI (Giao diện người dùng):**
 - LoginScreen: Màn hình đăng nhập để người dùng nhập thông tin đăng nhập.
 - ProfileScreen: Màn hình hiển thị thông tin cá nhân của người dùng và cho phép chỉnh sửa.
 - PostScreen: Màn hình cho phép người dùng đăng bài mới lên hệ thống.
 - NotificationScreen: Màn hình hiển thị các thông báo từ hệ thống.
- **Client (Khách hàng):**
 - AuthenticationModule: Module xử lý việc đăng nhập và đăng ký người dùng.
 - UserManagementModule: Module quản lý thông tin người dùng.
 - PostManagementModule: Module quản lý các bài đăng trên hệ thống.

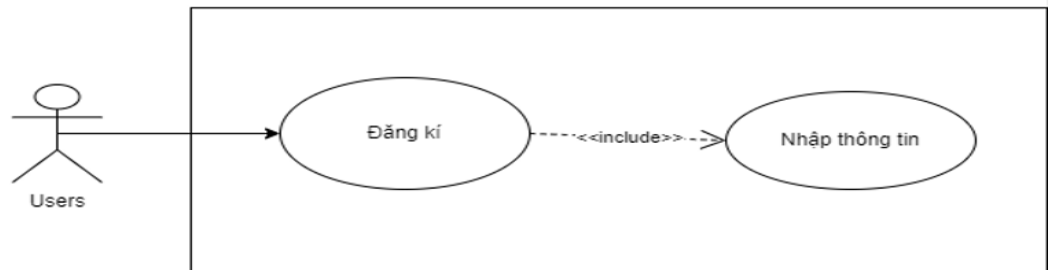
- NotificationModule: Module xử lý thông báo cho người dùng.
 - MessengerModule: Module xử lý tính năng nhắn tin giữa người dùng.
- API (Giao diện lập trình ứng dụng):
 - AuthAPI: API cung cấp các endpoint để xử lý đăng nhập và đăng ký.
 - UserAPI: API quản lý thông tin người dùng.
 - PostAPI: API quản lý các bài đăng trên hệ thống.
 - NotificationAPI: API quản lý và gửi thông báo.
 - MessengerAPI: API cho tính năng nhắn tin giữa người dùng.
 - Server (Máy chủ):
 - AuthService: Dịch vụ xử lý việc xác thực người dùng.
 - UserService: Dịch vụ quản lý thông tin người dùng.
 - PostService: Dịch vụ quản lý các bài đăng trên hệ thống.
 - NotificationService: Dịch vụ quản lý và gửi thông báo.
 - MessengerService: Dịch vụ xử lý tính năng nhắn tin giữa người dùng.
5. DB (Cơ sở dữ liệu):
- MariaDB: Hệ quản trị cơ sở dữ liệu MariaDB, lưu trữ dữ liệu người dùng, bài đăng, thông báo và tin nhắn.

2.2. Use case

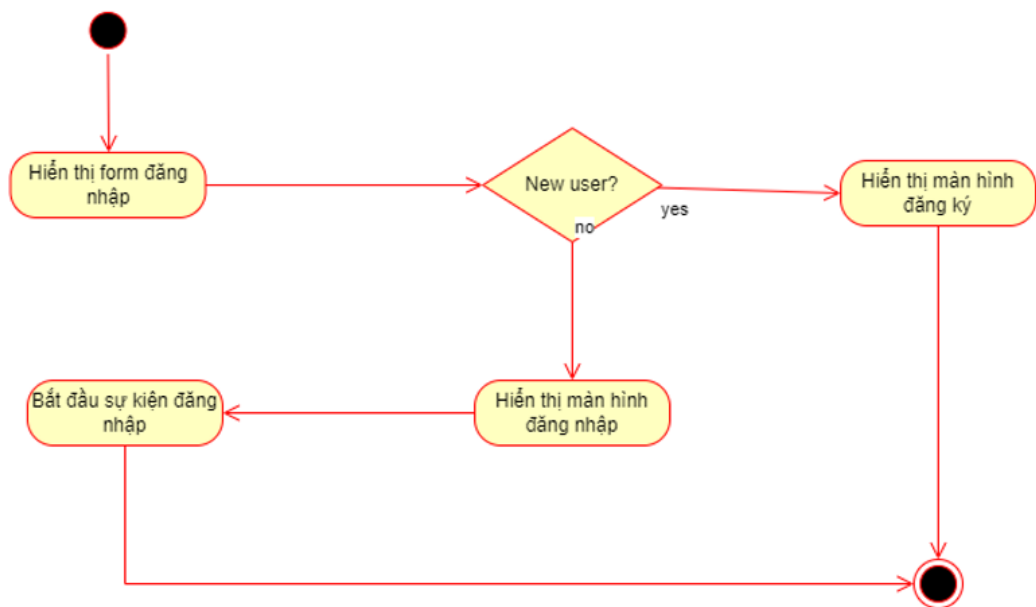


Hình 2: Mô hình use-case Tổng quát

2.2.1. Mô tả chi tiết use-case Đăng ký



Hình 3: Mô hình use-case đăng ký

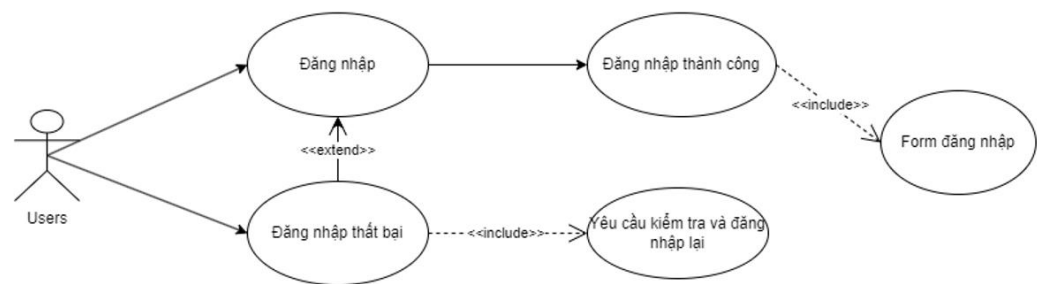


Hình 4: Activity Diagram cho Đăng ký

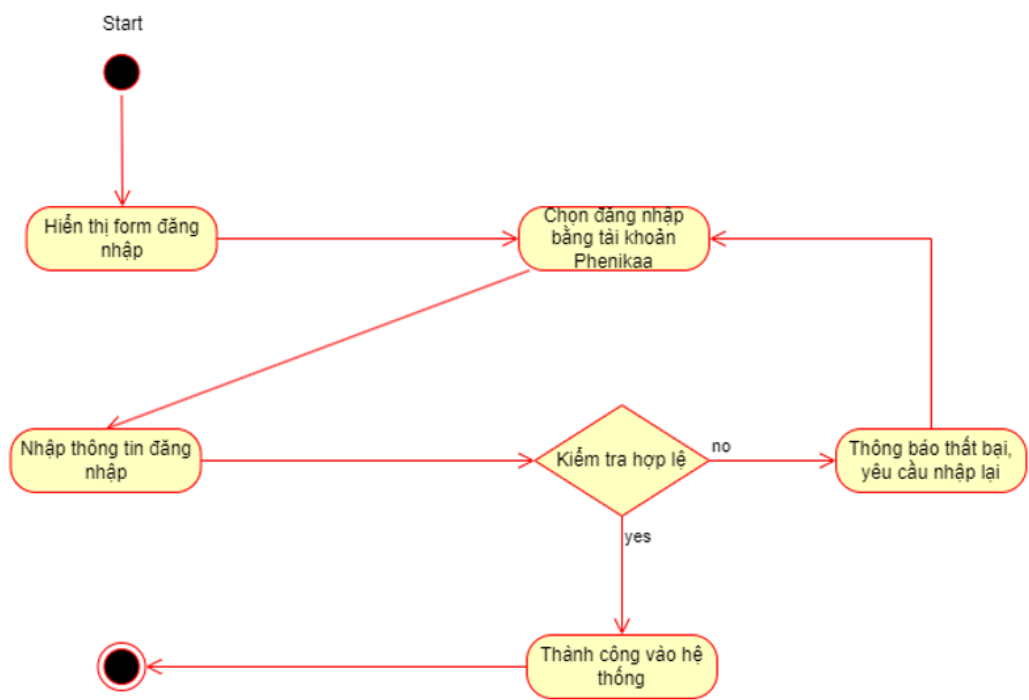
Use-case	Đăng ký
Tác nhân	Người dùng
Mô tả ngắn gọn	Use case này mô tả quá trình người dùng mới tạo tài khoản trên ứng dụng PU Connection bằng cách cung cấp thông tin cá nhân cần thiết và xác thực email.
Tiền điều kiện	Người dùng chưa có tài khoản trên ứng dụng PU Connection.
Chuỗi sự kiện chính	1. Người dùng mở ứng dụng PU Connection. 2. Người dùng chọn "Đăng ký" 3. Hệ thống hiển thị biểu mẫu đăng ký 4. Người dùng nhập các thông tin cần thiết (tên, email, mật khẩu) 5. Hệ thống kiểm tra tính hợp lệ của thông tin. 6. Hệ thống lưu thông tin người dùng vào cơ sở dữ liệu. 7. Hệ thống gửi email xác nhận đến người dùng. 8. Người dùng mở email và nhấn vào liên kết xác nhận. 9. Hệ thống thông báo đăng ký thành công.
Ngoại lệ	1.Email đã tồn tại 2.Thông tin không hợp lệ
Điều kiện đầu ra	Thông tin người dùng mới được lưu trữ an toàn trong cơ sở dữ liệu. Người dùng có thể sử dụng tài khoản mới để đăng nhập vào ứng dụng PU Connection.

Bảng 1: Use-case Đăng ký

2.2.2. Mô tả chi tiết use-case Đăng nhập



Hình 5: Mô hình use case đăng nhập

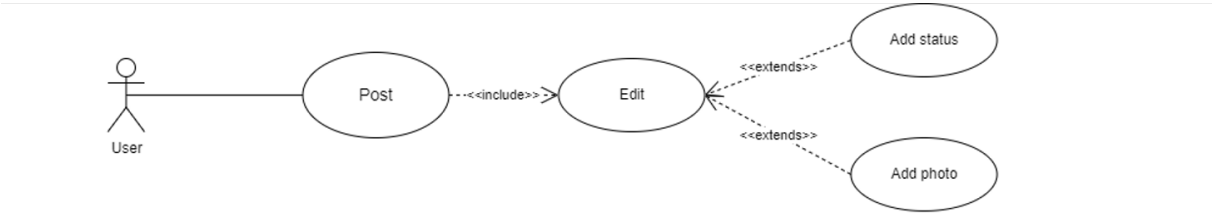


Hình 6: Activity Diagram cho Đăng nhập

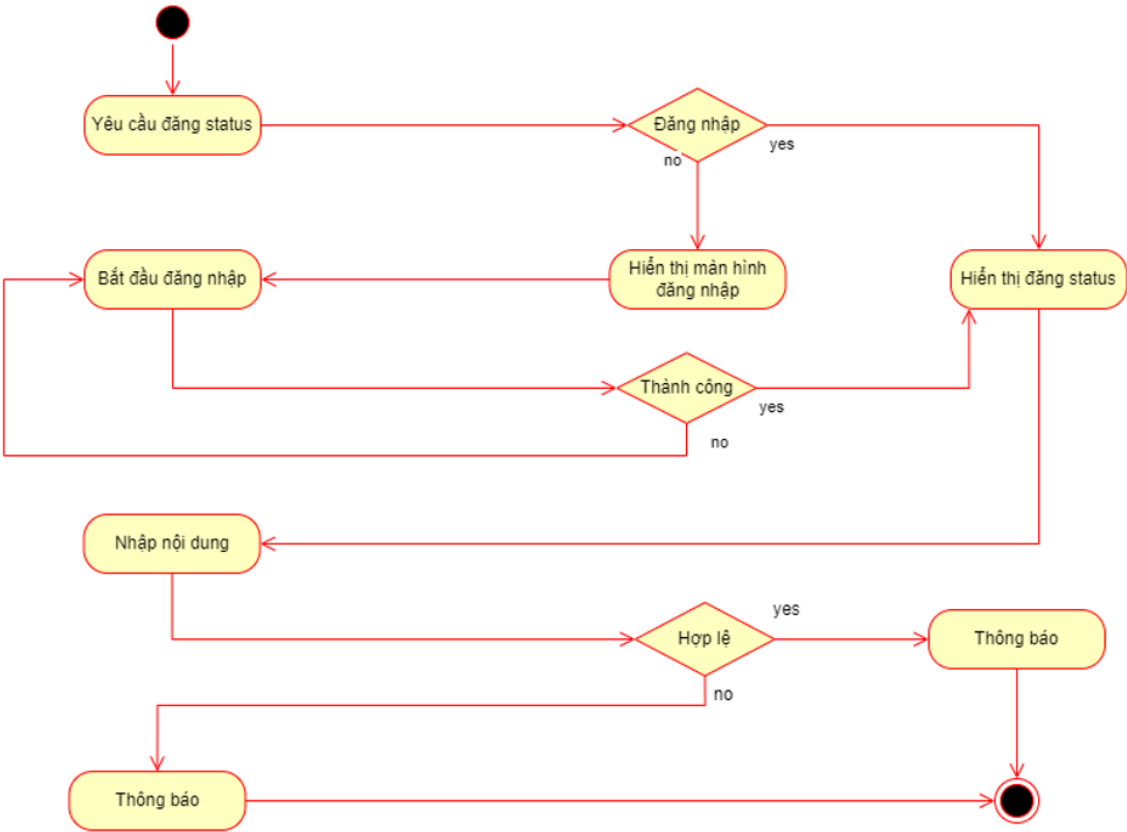
Use-case	Đăng nhập
Tác nhân	Người dùng
Mô tả ngắn gọn	Người dùng đăng nhập vào hệ thống, truy cập hệ thống với đúng quyền của tài khoản
Tiền điều kiện	Người dùng đã có tài khoản và đã xác thực email
Chuỗi sự kiện chính	1. Người dùng mở ứng dụng PU Connection 2. Người dùng chọn "Đăng nhập" 3. Người dùng nhập email và mật khẩu 4. Hệ thống kiểm tra dữ liệu: Nếu thỏa mãn thực hiện bước tiếp theo. Ngược lại, thông báo lỗi và quay lại bước 3 5. Hệ thống thông báo đăng nhập thành công và chuyển đến trang chính
Ngoại lệ	1. Email không tồn tại 2. Mật khẩu sai
Điều kiện đầu ra	Người dùng đăng nhập thành công và truy cập vào trang chính của ứng dụng PU Connection. Thông tin phiên đăng nhập được lưu trữ tạm thời để người dùng có thể duy trì phiên đăng nhập trong quá trình sử dụng ứng dụng.

Bảng 2: Use-case Đăng nhập

2.2.3. Mô tả chi tiết use-case đăng status



Hình 7 Use-case đăng status

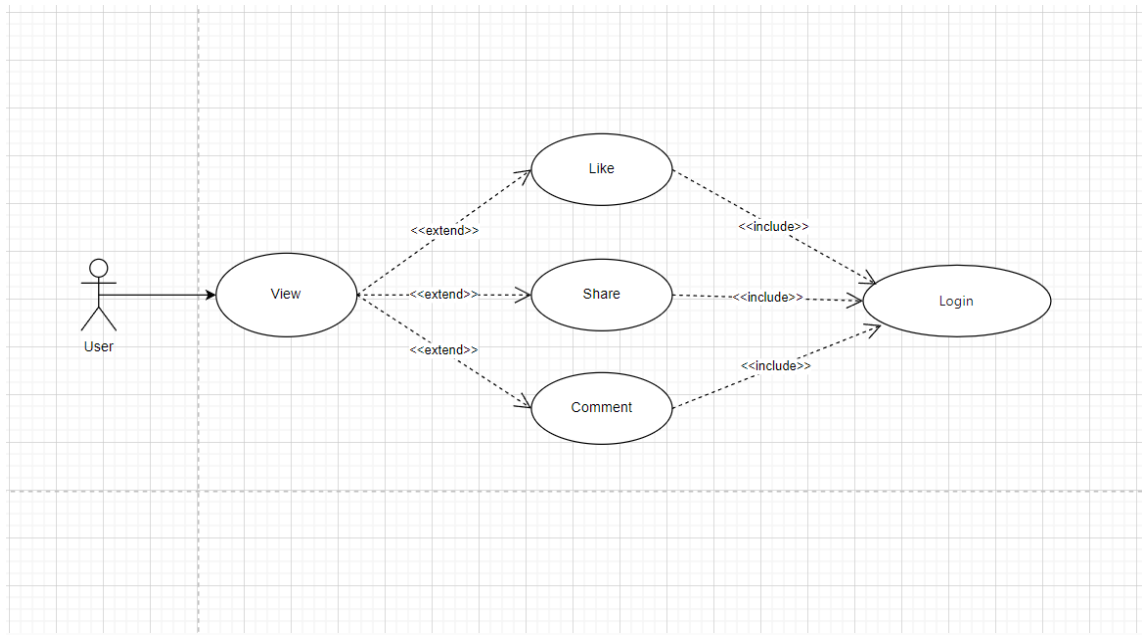


Hình 8: Activity Diagram cho đăng status

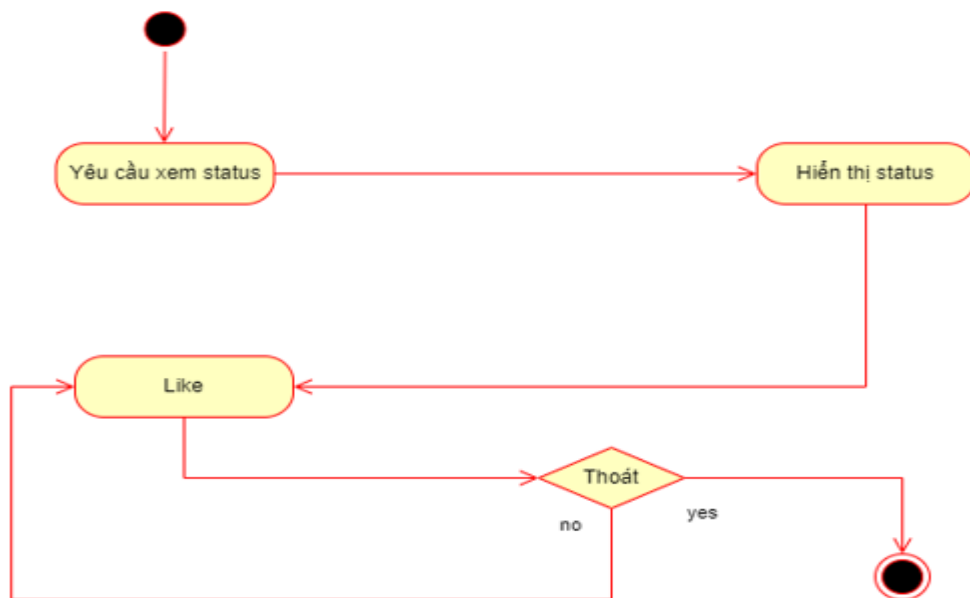
Use-case	Đăng status
Tác nhân	Người dùng
Mô tả ngắn gọn	Use case này mô tả quá trình người dùng đăng một trạng thái mới lên ứng dụng PU Connection để chia sẻ với cộng đồng.
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống.
Chuỗi sự kiện chính	1. Người dùng mở ứng dụng PU Connection. 2. Người dùng chọn nút có biểu tượng "Đăng status". 3. Người dùng nhập nội dung status (Có thể đăng kèm ảnh hoặc video) 4. Người dùng nhấn nút "Post". 5. Hệ thống lưu trạng thái vào cơ sở dữ liệu. 6. Hệ thống hiển thị trạng thái mới trên tường của người dùng và thông báo đăng thành công.
Ngoại lệ	1. Nội dung status rỗng 2. Nội dung không hợp lệ
Điều kiện đầu ra	Trạng thái mới của người dùng được lưu trữ thành công trong cơ sở dữ liệu và hiển thị trên tường.

Bảng 3: Use-case đăng status

2.2.4. Mô tả chi tiết use-case xem status



Hình 9. Use-case xem status

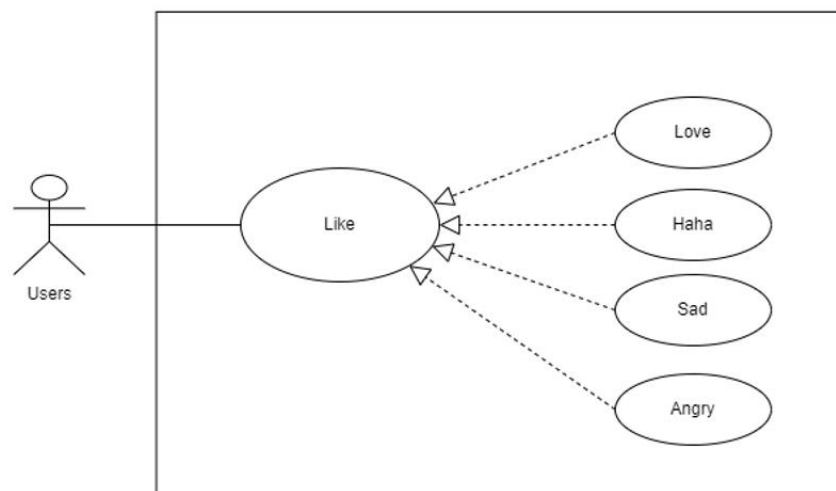


Hình 10: Activity Diagram cho xem status

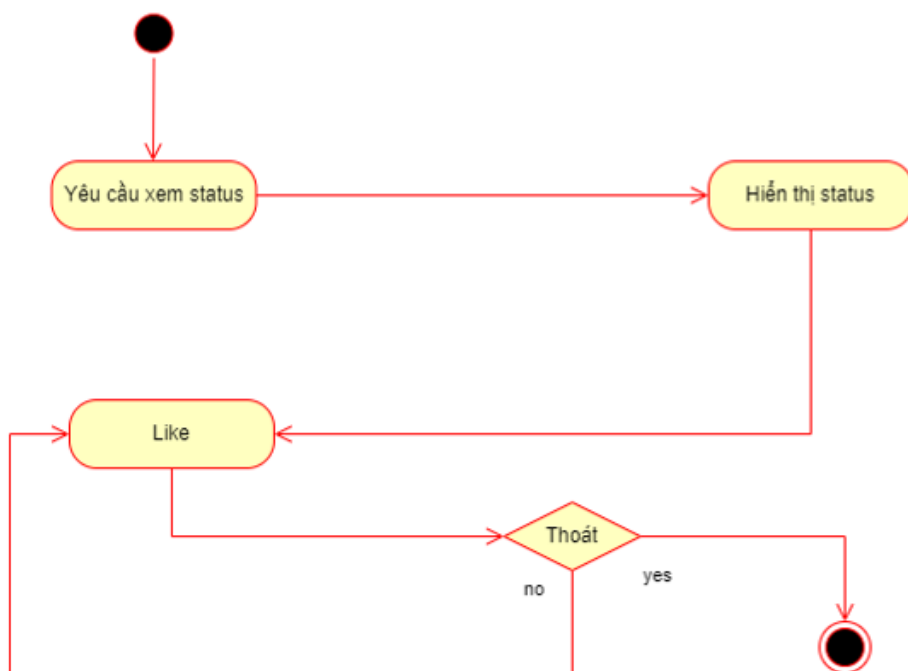
Use-case	Xem status
Tác nhân	Người dùng
Mô tả ngắn gọn	Use case này mô tả quá trình người dùng xem và tương tác với các status được đăng trên tường hoặc trong hồ sơ cá nhân của người dùng khác.
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống
Chuỗi sự kiện chính	1. Người dùng mở ứng dụng PU Connection. 2. Người dùng chọn tường (news feed) hoặc hồ sơ của một người dùng khác 3. Hệ thống tải và hiển thị các status từ cơ sở dữ liệu. 4. Người dùng cuộn để xem các status khác nhau.
Ngoại lệ	Lỗi tải dữ liệu
Điều kiện đầu ra	Các status được hiển thị đầy đủ và chính xác trên giao diện người dùng.

Bảng 4: Use-case xem status

2.2.5. Mô tả chi tiết use-case Tương tác bài viết (Like status)



Hình 11: Mô hình use-case like status



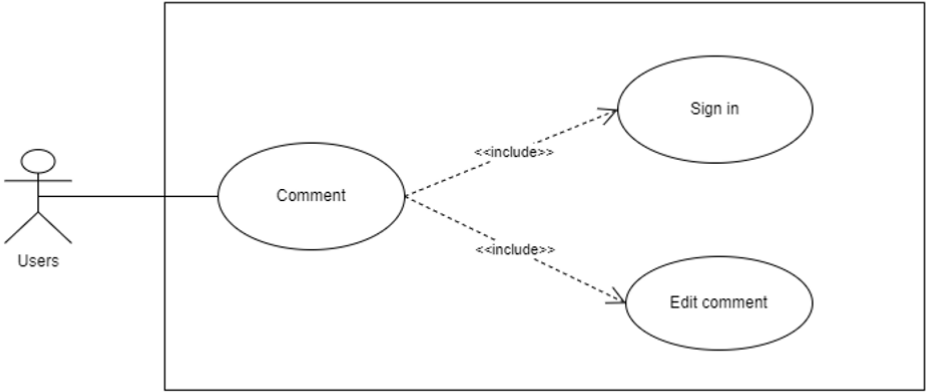
Hình 12: Activity Diagram cho Tương tác bài viết (Thả tim)

Use-case	Like Status
----------	-------------

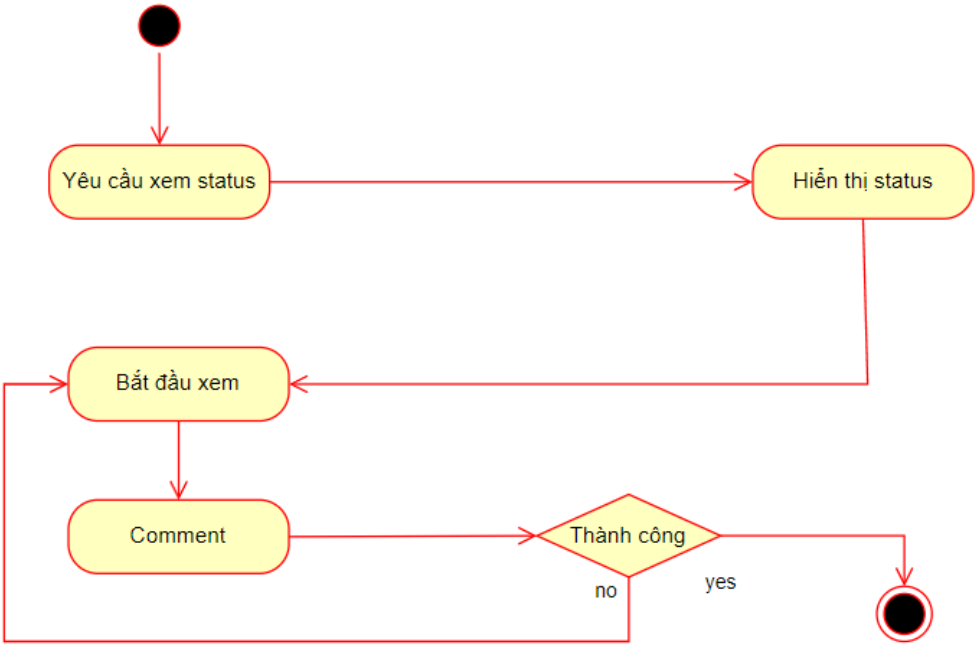
Tác nhân	Người dùng
Mô tả ngắn gọn	Use case này mô tả quá trình người dùng thích một status trên ứng dụng PU Connection.
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống. Người dùng đang xem một status.
Chuỗi sự kiện chính	1. Người dùng chọn biểu tượng "Thích" (Like) trên status. 2. Hệ thống cập nhật thông tin status trong cơ sở dữ liệu. 3. Hệ thống cập nhật giao diện để hiển thị status đã được thích.
Ngoại lệ	Lỗi cập nhật dữ liệu
Điều kiện đầu ra	Status được cập nhật với số lượt thích mới và hiển thị người dùng đã thích status.

Bảng 5: Use-case tương tác bài viết (Like status)

2.2.6. Mô tả chi tiết use-case Bình luận



Hình 13: Mô hình use-case bình luận

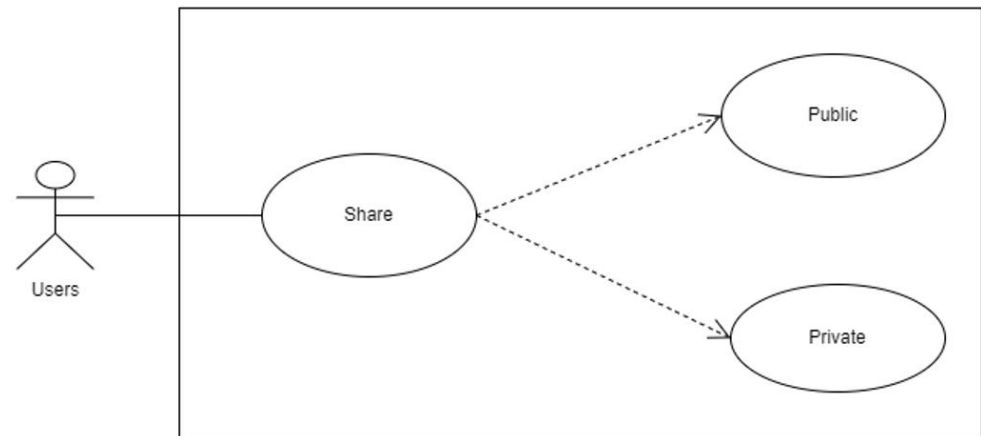


Hình 14: Activity Diagram cho Bình luận

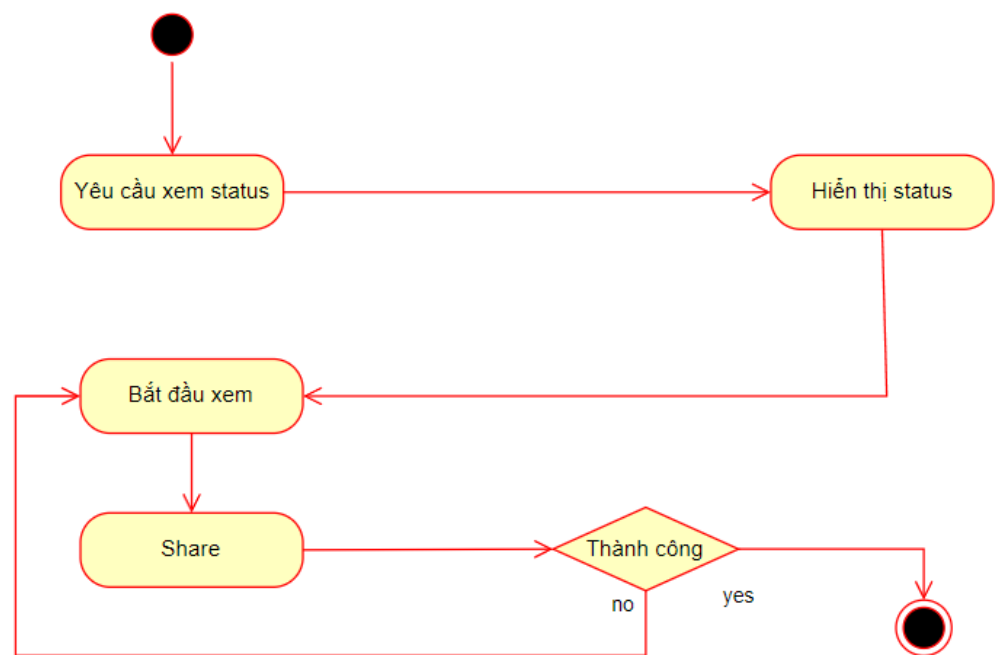
Use-case	Bình luận status
Tác nhân	Người dùng
Mô tả ngắn gọn	Use case này mô tả quá trình người dùng bình luận một status trên ứng dụng PU Connection.
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống. Người dùng đang xem một status.
Chuỗi sự kiện chính	1. Người dùng chọn biểu tượng "Bình luận" (Comment) trên status. 2. Người dùng nhập nội dung bình luận. 3. Hệ thống kiểm tra tính hợp lệ của nội dung bình luận. 4. Hệ thống lưu bình luận vào cơ sở dữ liệu. 5. Hệ thống cập nhật giao diện để hiển thị bình luận mới.
Ngoại lệ	Nội dung bình luận rỗng Lỗi cập nhật dữ liệu
Điều kiện đầu ra	Bình luận mới được lưu trữ thành công trong cơ sở dữ liệu và hiển thị dưới status.

Bảng 6: Use-case Bình luận

2.2.7. Mô tả chi tiết use-case share status



Hình 15: Mô hình use-case share status

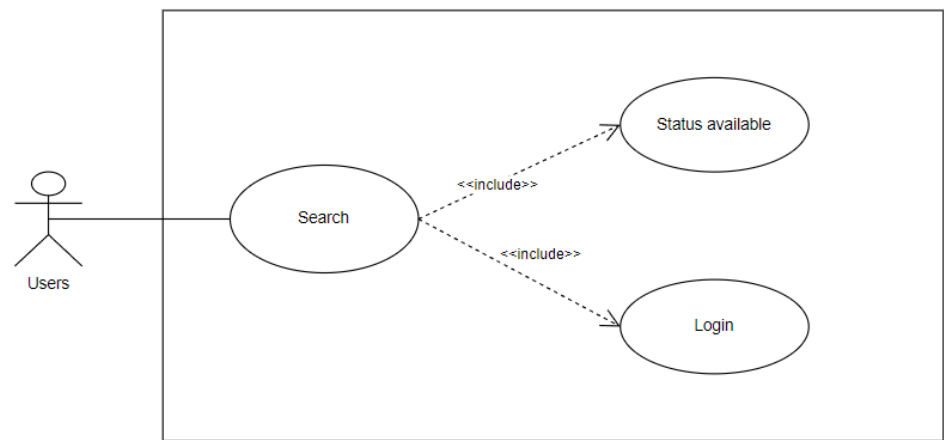


Hình 16: Activity Diagram cho Share status

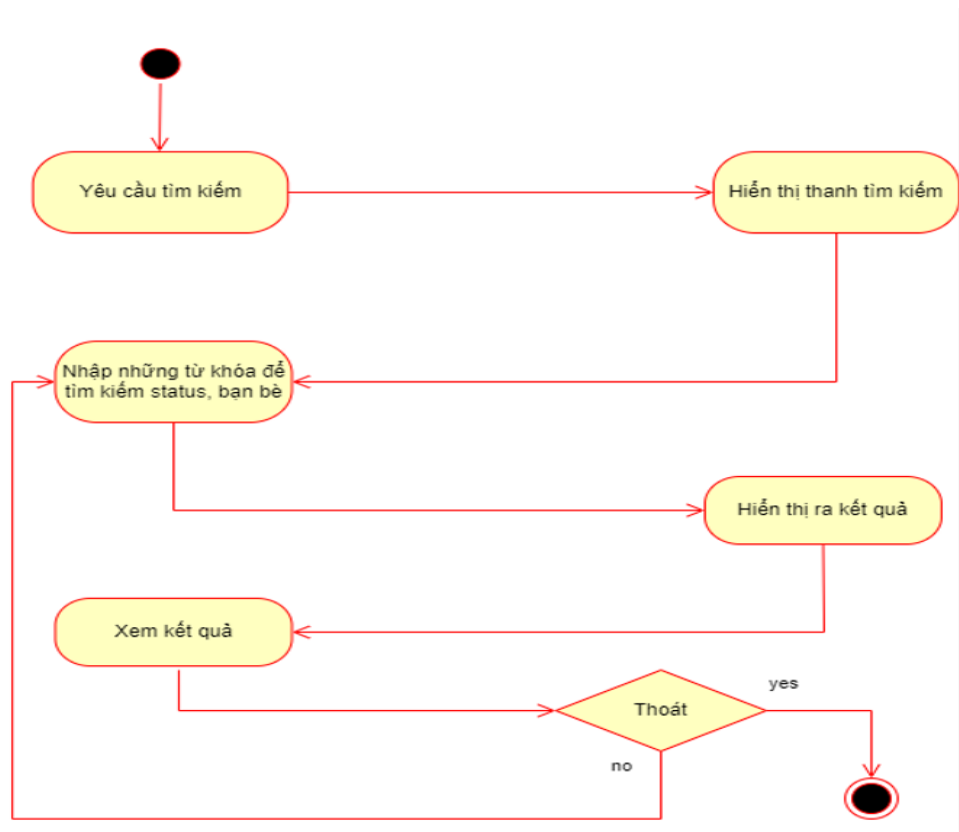
Use-case	Share status
Tác nhân	Người dùng
Mô tả ngắn gọn	Use case này mô tả quá trình người dùng chia sẻ một status trên ứng dụng PU Connection.
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống. Người dùng đang xem một status.
Chuỗi sự kiện chính	1. Người dùng chọn biểu tượng "Chia sẻ" (Share) trên status. 2. Người dùng chọn nơi chia sẻ (ví dụ: trên tường của mình, trong một nhóm, hoặc qua tin nhắn). 3. Người dùng nhập nội dung bổ sung (nếu có) và xác nhận chia sẻ. 4. Hệ thống lưu thông tin chia sẻ vào cơ sở dữ liệu. 5. Hệ thống cập nhật giao diện để hiển thị status đã được chia sẻ.
Ngoại lệ	Lỗi chia sẻ.
Điều kiện đầu ra	Status được chia sẻ thành công và hiển thị tại nơi người dùng đã chọn.

Bảng 7: Use-case share status

2.2.8. Mô tả chi tiết use-case Tìm kiếm status



Hình 17: Mô hình use-case tìm kiếm status

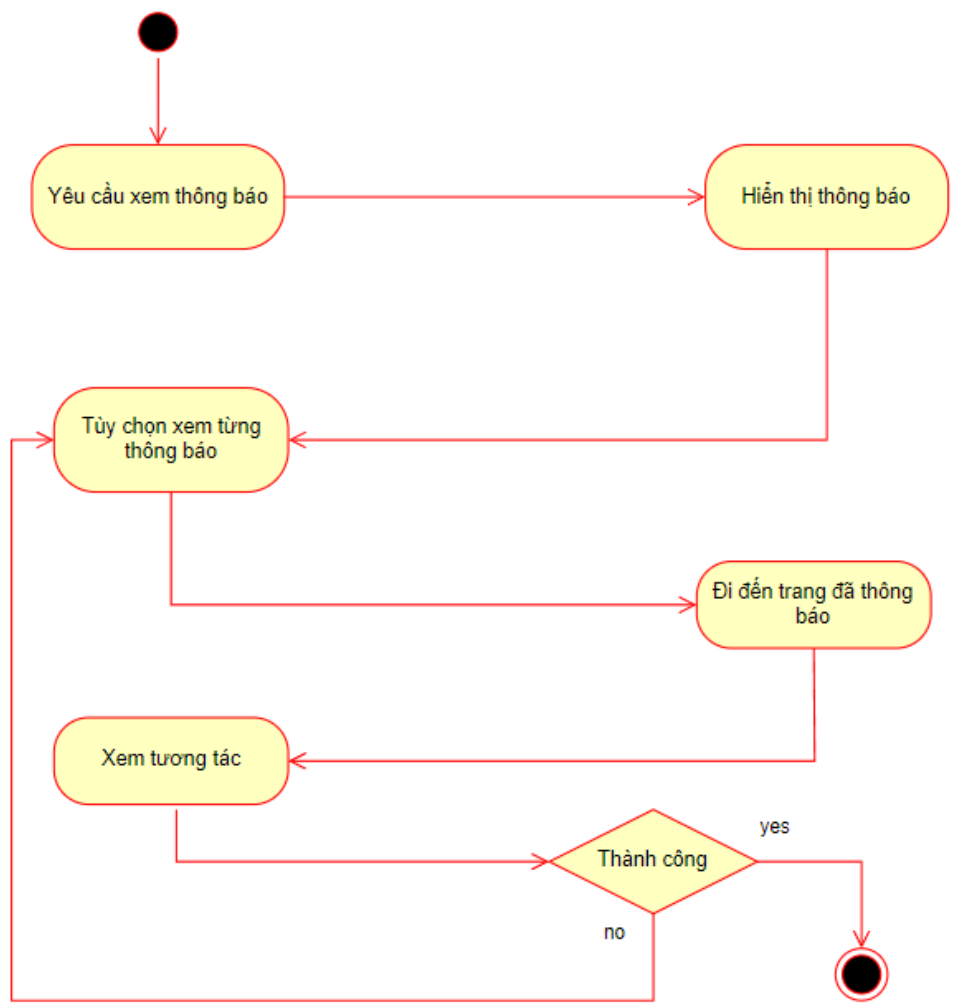


Hình 18: Activity Diagram cho Tìm kiếm

Use-case	Tìm kiếm status
Tác nhân	Người dùng
Mô tả ngắn gọn	Use case này mô tả quá trình người dùng tìm kiếm các status trên ứng dụng PU Connection.
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống.
Chuỗi sự kiện chính	1. Người dùng mở tính năng tìm kiếm trên ứng dụng PU Connection. 2. Người dùng nhập từ khóa hoặc cụm từ cần tìm kiếm. 3. Người dùng nhấn nút "Tìm kiếm" hoặc nhập xong từ khóa trên bàn phím. 4. Hệ thống xử lý yêu cầu tìm kiếm. 5. Hệ thống trả về kết quả tìm kiếm cho người dùng.
Ngoại lệ	Hệ thống thông báo lỗi "Không tìm thấy kết quả phù hợp" nếu không có status nào phù hợp với từ khóa hoặc cụm từ tìm kiếm.
Điều kiện đầu ra	Các status phù hợp với từ khóa hoặc cụm từ tìm kiếm được hiển thị đúng trên giao diện người dùng.

Bảng 8: Use-case Tìm kiếm

2.2.9. Mô tả chi tiết use-case Thông báo



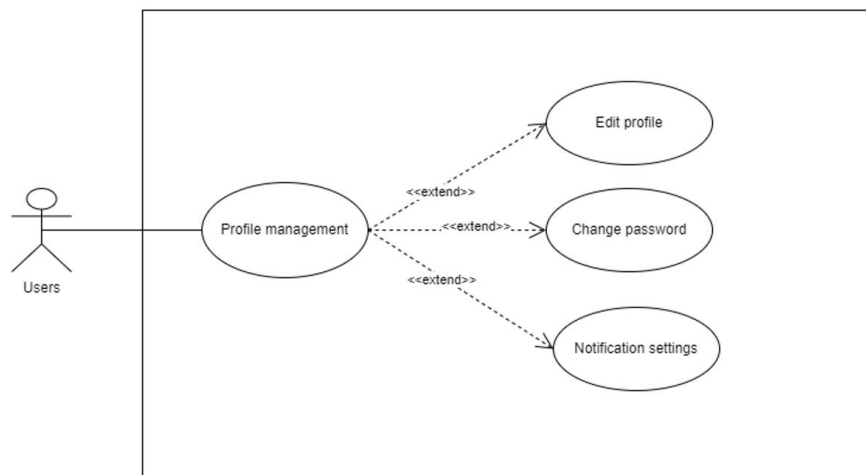
Hình 19: Activity Diagram cho Thông báo

Use-case	Nội dung
Tên use-case	Thông báo
Mô tả	Người dùng xem các thông báo trong ứng dụng
Tác nhân	Người dùng, Hệ thống
Điều kiện kích hoạt	Người dùng đang đăng nhập và sử dụng ứng dụng
Điều kiện đầu vào	Giao diện người dùng đang ở màn hình Thông báo Thiết bị người dùng được kết nối Internet

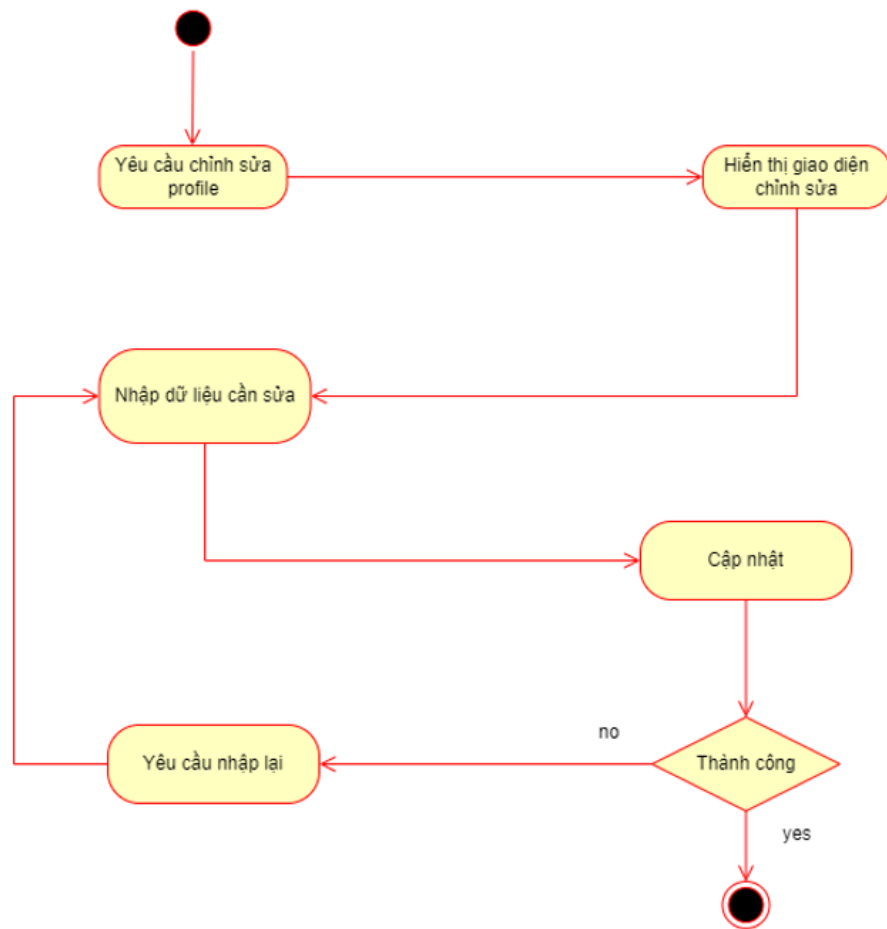
Luồng cơ bản	1. Ở giao diện chính sau khi đăng nhập, người dùng sẽ chọn icon "Notification" ở thanh BottomAppBar 2. Hệ thống sẽ hiển thị các thông báo mà người dùng nhận
Luồng thay thế	Không
Luồng ngoại lệ	2.1. Nếu không có thông báo nào để hiển thị, hệ thống hiển thị màn hình thông báo "Không có thông báo nào".

Bảng 9: Use-case Thông báo

2.2.10. Mô tả chi tiết use-case Quản lý profile



Hình 20: Mô hình use-case quản lý profile



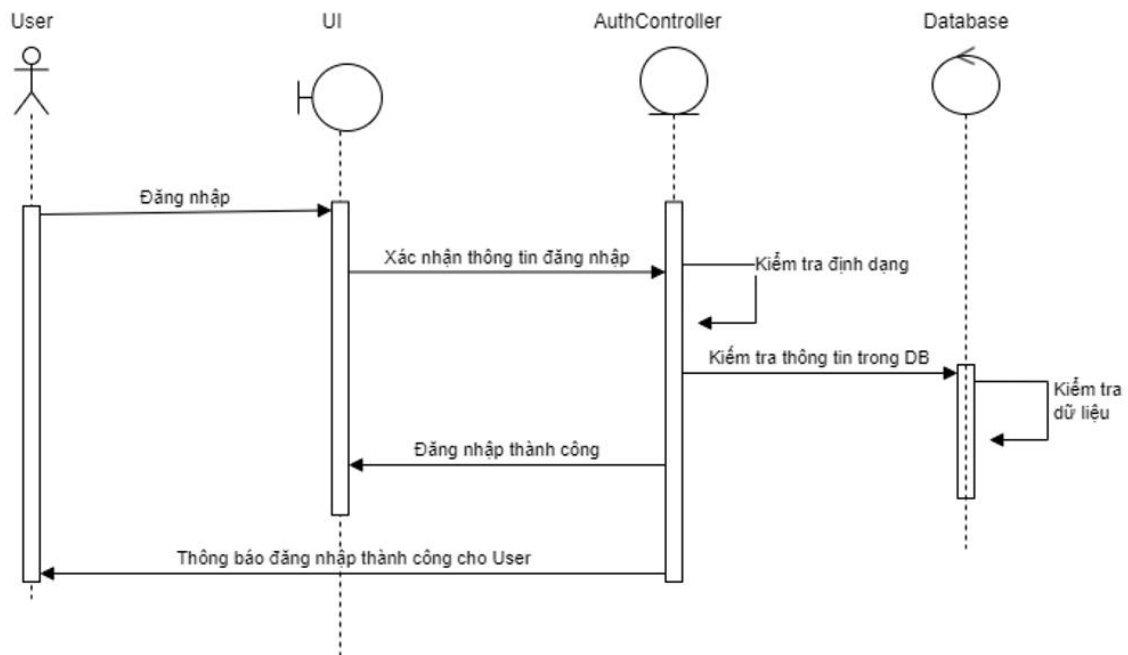
Hình 21: Activity Diagram cho Chỉnh sửa profile

Use-case	Quản lý Profile
Tác nhân	Người dùng
Mô tả ngắn gọn	Use case này mô tả quá trình người dùng quản lý thông tin cá nhân và cài đặt tài khoản trên ứng dụng PU Connection.
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống.
Chuỗi sự kiện chính	1. Người dùng mở màn hình quản lý profile trong ứng dụng. 2. Người dùng chọn sửa đổi thông tin cá nhân. 3. Người dùng cập nhật thông tin cá nhân và lưu thay đổi. 4. Người dùng thay đổi cài đặt bảo mật hoặc cài đặt khác của tài khoản. 5. Người dùng thực hiện các thay đổi cài đặt và lưu.
Ngoại lệ	Lỗi cập nhật thông tin
Điều kiện đầu ra	Thông tin cá nhân và cài đặt tài khoản của người dùng được cập nhật thành công và lưu lại trong hệ thống.

Bảng 10: Use-case Chỉnh sửa thông tin cá nhân

2.3. Các biểu đồ tuần tự

2.3.1. Biểu đồ tuần tự Đăng nhập

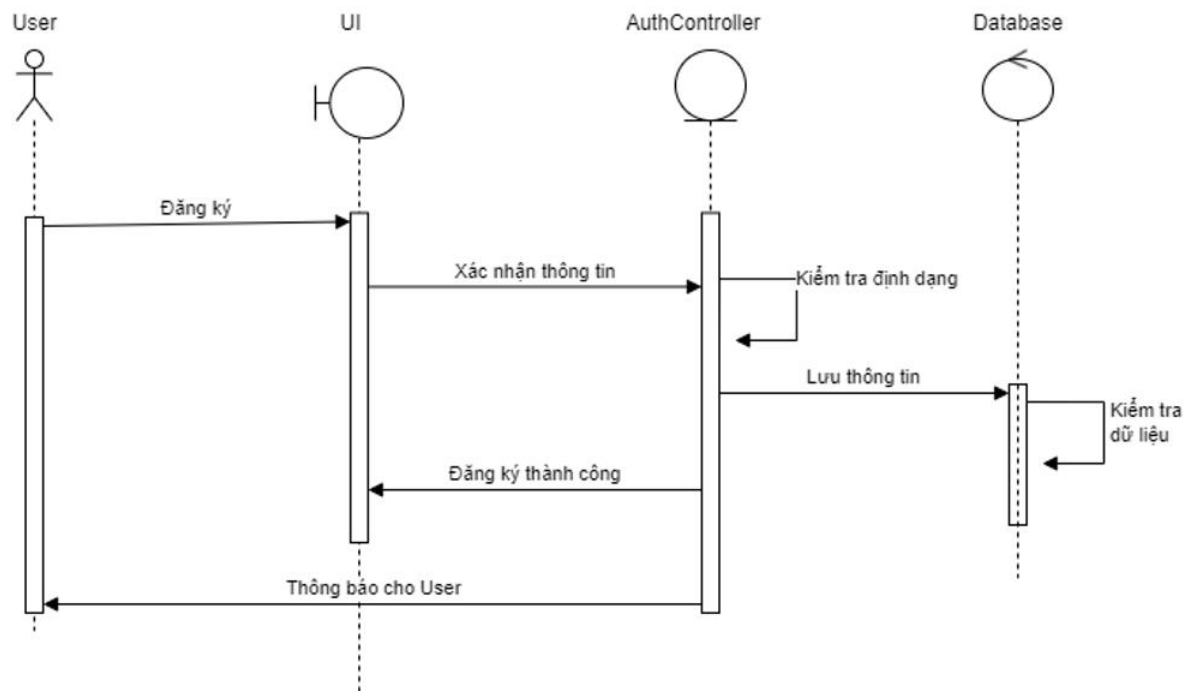


Hình 22: Biểu đồ tuần tự Đăng nhập

Mô tả biểu đồ tuần tự đăng nhập:

1. User nhập email và mật khẩu trên giao diện đăng nhập.
2. UI gửi yêu cầu đăng nhập tới Client.
3. Client gọi Auth API với thông tin đăng nhập.
4. Auth API chuyển yêu cầu đến Auth Service.
5. Auth Service xác thực thông tin người dùng trong MariaDB.
6. Nếu thông tin hợp lệ, Auth Service tạo và trả về token đăng nhập.
7. Auth API gửi token về Client.
8. Client cập nhật trạng thái đăng nhập và chuyển tới màn hình chính.

2.3.2. Biểu đồ tuần tự Đăng ký

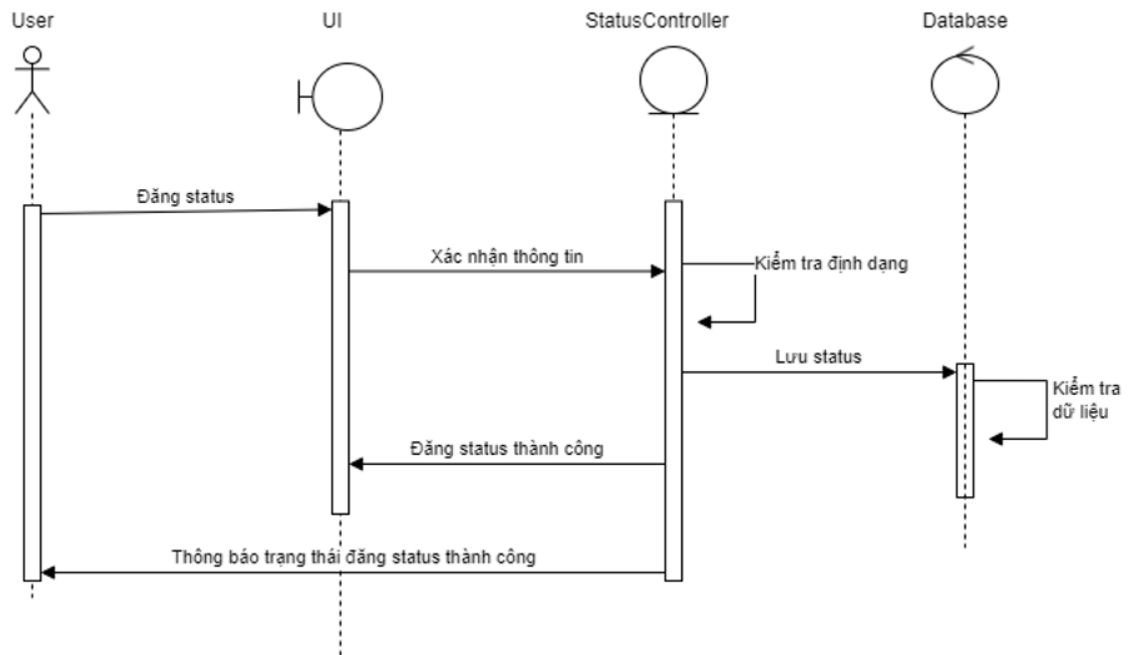


Hình 23: Biểu đồ tuần tự Đăng kí

Mô tả biểu đồ tuần tự đăng ký

1. User nhập thông tin đăng ký trên giao diện đăng ký.
2. UI gửi yêu cầu đăng ký tới Client.
3. Client gọi Auth API với thông tin đăng ký.
4. Auth API chuyển yêu cầu đến Auth Service.
5. Auth Service lưu thông tin người dùng vào MariaDB.
6. Auth API trả kết quả về Client.
7. Client thông báo kết quả đăng ký trên UI.

2.3.3. Biểu đồ tuần tự Đăng status

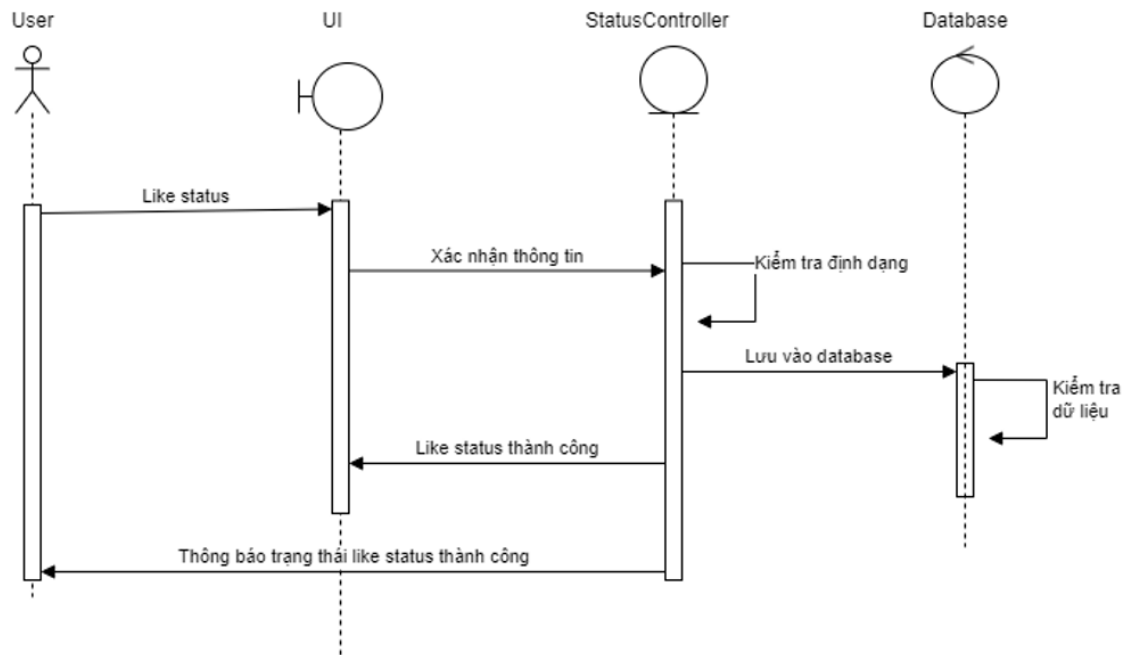


Hình 24: Biểu đồ tuần tự Đăng status

Mô tả biểu đồ tuần tự đăng status

1. User nhập nội dung status trên giao diện đăng bài.
2. UI gửi yêu cầu đăng bài tới Client.
3. Client gọi Post API với thông tin bài đăng.
4. Post API chuyển yêu cầu tới Post Service.
5. Post Service lưu thông tin bài đăng vào MariaDB.
6. Post API trả kết quả về Client.
7. Client hiển thị bài đăng mới trên UI.

2.3.4. Biểu đồ tuần tự Tương tác bài viết (Like status)

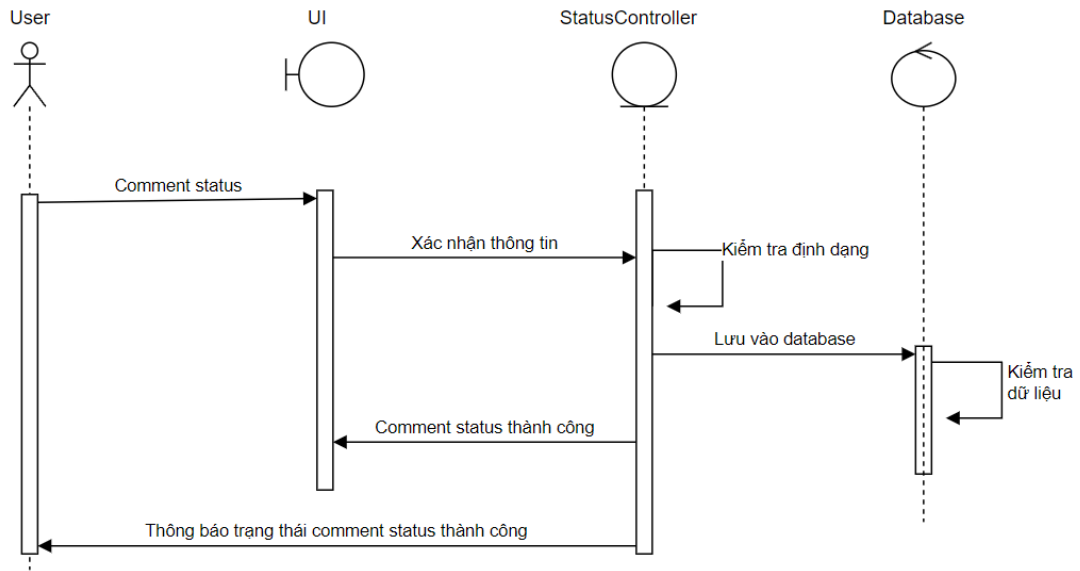


Hình 25: Biểu đồ tuần tự Tương tác bài viết (Like status)

Mô tả biểu đồ tuần tự like status

1. User nhấn nút "Like" trên một status.
2. UI gửi yêu cầu thích bài đăng tới Client.
3. Client gọi Post API với thông tin bài đăng cần thích.
4. Post API chuyển yêu cầu tới Post Service.
5. Post Service cập nhật thông tin lượt thích trong MariaDB.
6. Post API trả kết quả về Client.
7. Client cập nhật trạng thái lượt thích trên UI.

2.3.5. Biểu đồ tuần tự comment status

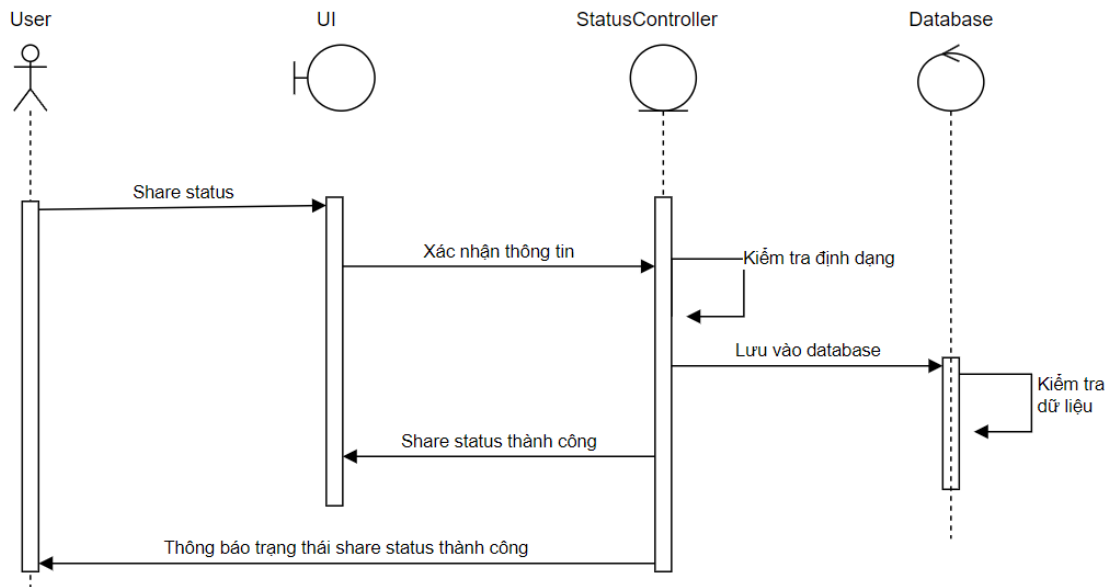


Hình 26: Biểu đồ tuần tự comment status

Mô tả biểu đồ tuần tự comment status

1. User nhập nội dung bình luận trên một status.
2. UI gửi yêu cầu bình luận tới Client.
3. Client gọi Comment API với thông tin bình luận.
4. Comment API chuyển yêu cầu tới Comment Service.
5. Comment Service lưu thông tin bình luận vào MariaDB.
6. Comment API trả kết quả về Client.
7. Client hiển thị bình luận mới trên UI.

2.3.6. Biểu đồ tuần tự share status

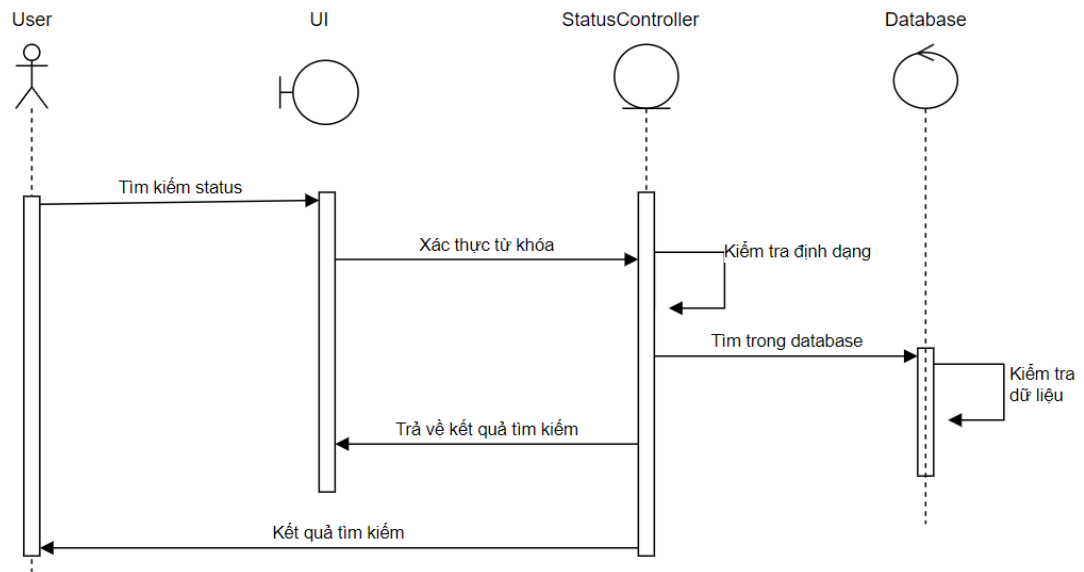


Hình 27: Biểu đồ tuần tự share status

Mô tả biểu đồ tuần tự share status

1. User nhấn nút "Share" trên một status.
2. UI gửi yêu cầu chia sẻ bài đăng tới Client.
3. Client gọi Post API với thông tin bài đăng cần chia sẻ.
4. Post API chuyển yêu cầu tới Post Service.
5. Post Service cập nhật thông tin lượt chia sẻ trong MariaDB.
6. Post API trả kết quả về Client.
7. Client hiển thị bài đăng đã chia sẻ trên UI.

2.3.7. Biểu đồ tuần tự Tìm kiếm

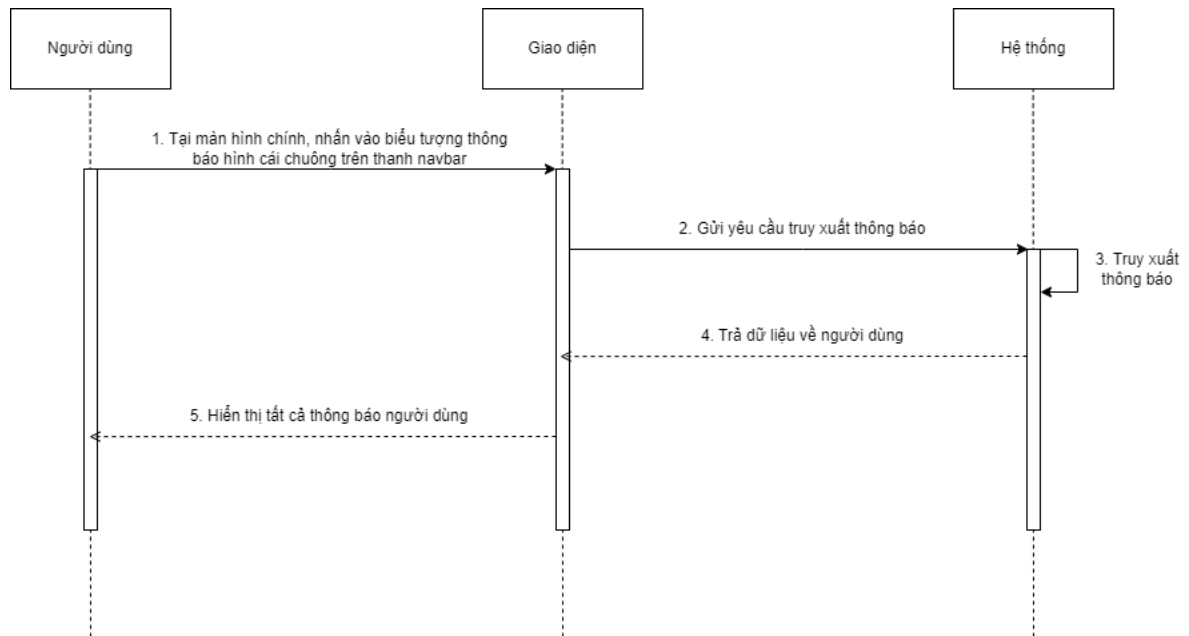


Hình 28: Biểu đồ tuần tự Tìm kiếm

Mô tả biểu đồ tuần tự tìm kiếm

1. User nhập từ khóa tìm kiếm trên giao diện tìm kiếm.
2. UI gửi yêu cầu tìm kiếm tới Client.
3. Client gọi Search API với từ khóa tìm kiếm.
4. Search API chuyển yêu cầu tới Search Service.
5. Search Service truy vấn MariaDB để lấy danh sách kết quả.
6. Search API trả kết quả về Client.
7. Client hiển thị kết quả tìm kiếm trên UI.

2.3.8. Biểu đồ tuần tự Thông báo

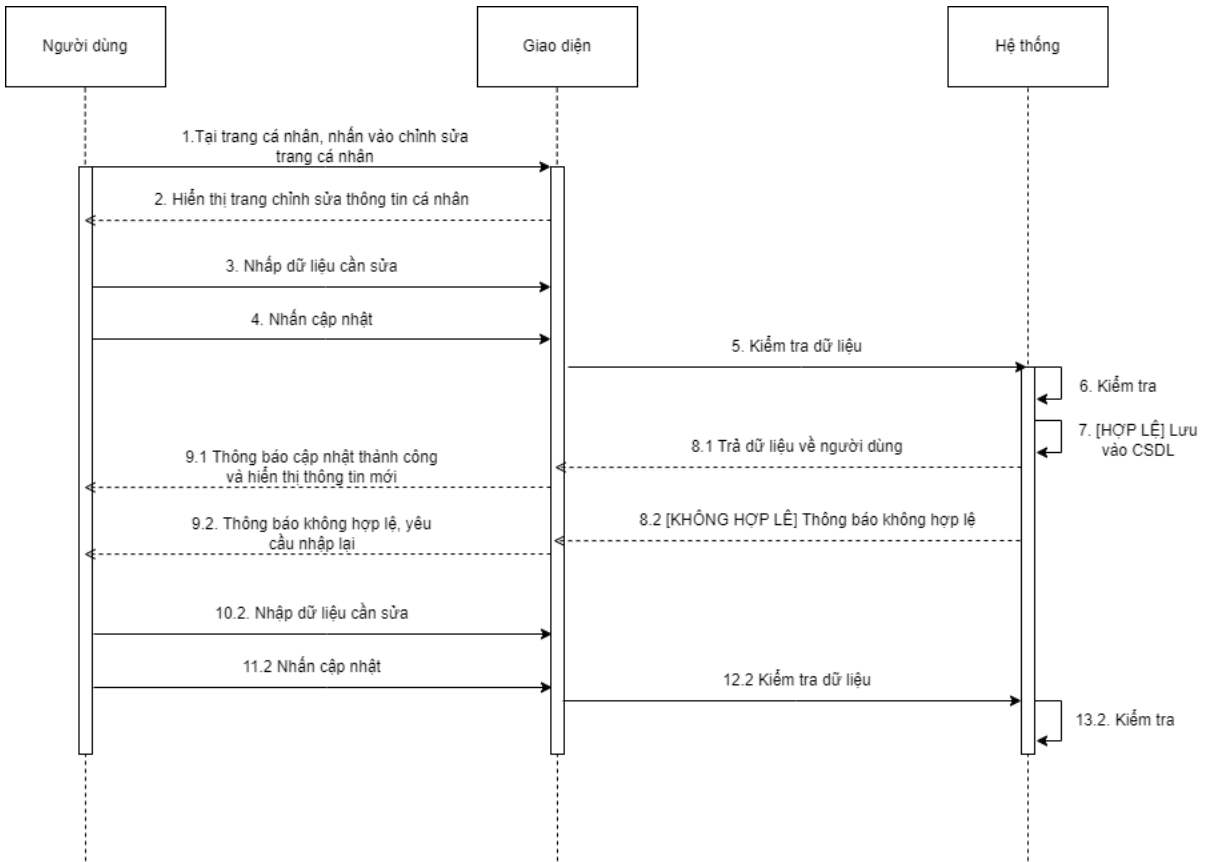


Hình 29: Biểu đồ tuần tự Thông báo

Mô tả biểu đồ tuần tự thông báo

1. User thực hiện một hành động cần thông báo (ví dụ: like, comment).
2. UI gửi yêu cầu thông báo tới Client.
3. Client gọi Notification API với thông tin thông báo.
4. Notification API chuyển yêu cầu tới Notification Service.
5. Notification Service lưu thông tin thông báo vào MariaDB.
6. Notification API trả kết quả về Client.
7. Client hiển thị thông báo mới trên UI.

2.3.9. Biểu đồ tuần tự **Chỉnh sửa thông tin cá nhân**



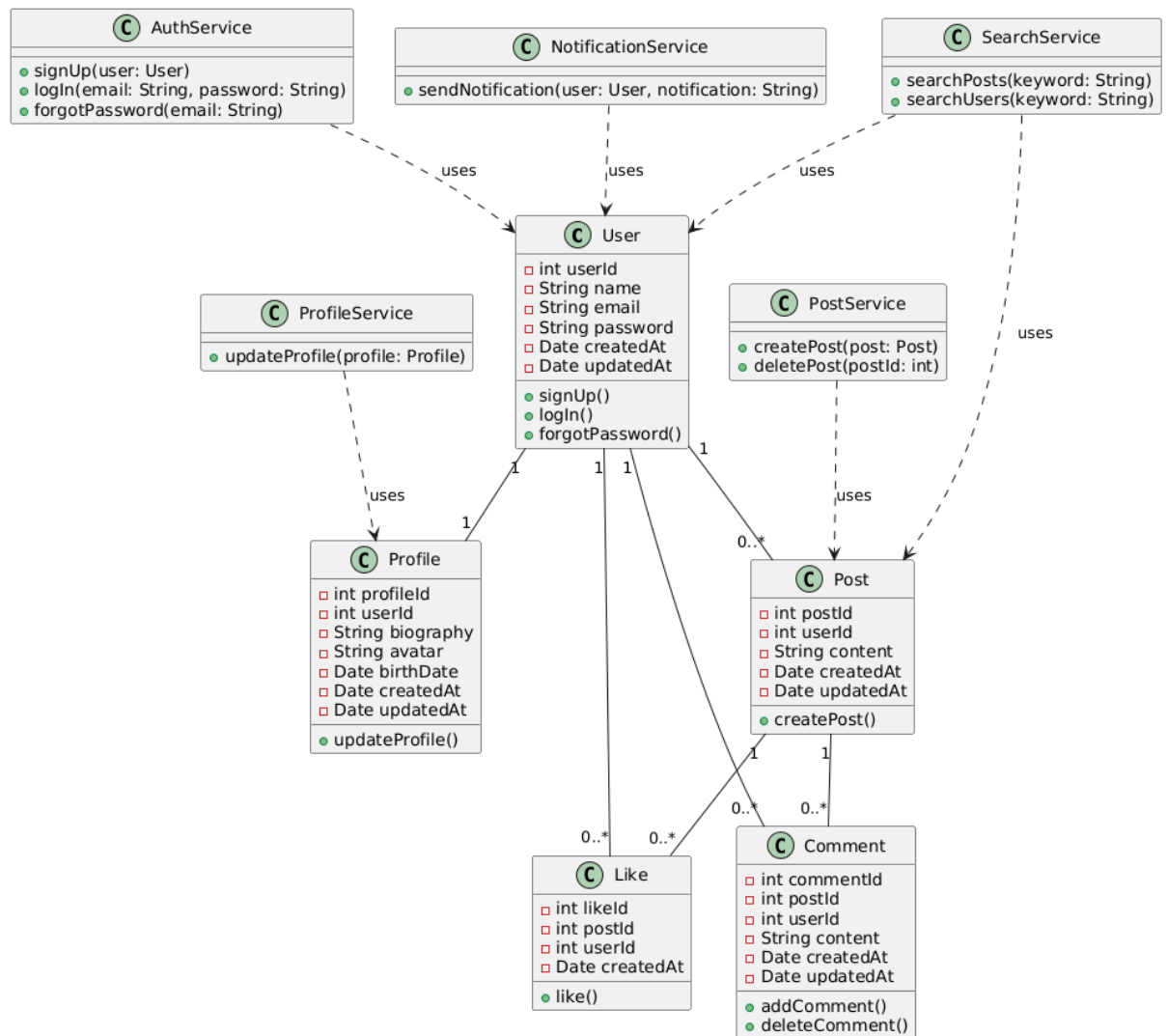
Hình 30: Biểu đồ tuần tự Chỉnh sửa thông tin cá nhân

Mô tả biểu đồ tuần tự chỉnh sửa profile

1. User cập nhật thông tin cá nhân trên giao diện profile.
2. UI gửi yêu cầu chỉnh sửa profile tới Client.
3. Client gọi Profile API với thông tin mới.
4. Profile API chuyển yêu cầu tới Profile Service.
5. Profile Service cập nhật thông tin người dùng trong MariaDB.
6. Profile API trả kết quả về Client.
7. Client cập nhật thông tin mới trên UI.

2.4. Mô hình thiết kế lớp

2.4.1. Xác định các lớp



Hình 31: Sơ đồ thiết kế lớp

Mô tả sơ đồ lớp:

- Các lớp và thuộc tính, phương thức

1. User

- Thuộc tính:

- `userId (int)` : Mã định danh duy nhất của người dùng.
- `name (String)` : Tên của người dùng.
- `email (String)` : Email của người dùng.
- `password (String)` : Mật khẩu của người dùng.

- `createdAt (Date)` : Thời gian tạo tài khoản.
- `updatedAt (Date)` : Thời gian cập nhật thông tin tài khoản lần cuối.
- Phương thức:
 - `signUp()` : Đăng ký tài khoản người dùng mới.
 - `login()` : Đăng nhập vào hệ thống.
 - `forgotPassword()` : Quên mật khẩu và yêu cầu đặt lại mật khẩu.

2. Profile

- Thuộc tính:
 - `profileId (int)` : Mã định danh duy nhất của hồ sơ.
 - `userId (int)` : Mã người dùng liên kết với hồ sơ.
 - `biography (String)` : Tiểu sử của người dùng.
 - `avatar (String)` : Ảnh đại diện của người dùng.
 - `birthDate (Date)` : Ngày sinh của người dùng.
 - `createdAt (Date)` : Thời gian tạo hồ sơ.
 - `updatedAt (Date)` : Thời gian cập nhật hồ sơ lần cuối.
- Phương thức:
 - `updateProfile()` : Cập nhật thông tin hồ sơ người dùng.

3. Post

- Thuộc tính:
 - `postId (int)` : Mã định danh duy nhất của bài đăng.
 - `userId (int)` : Mã người dùng tạo bài đăng.
 - `content (String)` : Nội dung của bài đăng.
 - `createdAt (Date)` : Thời gian tạo bài đăng.
 - `updatedAt (Date)` : Thời gian cập nhật bài đăng lần cuối.
- Phương thức:
 - `createPost()` : Đăng bài viết mới.

4. Comment

- Thuộc tính:
 - `commentId (int)` : Mã định danh duy nhất của bình luận.
 - `postId (int)` : Mã bài đăng mà bình luận thuộc về.
 - `userId (int)` : Mã người dùng tạo bình luận.

- ``content (String)``: Nội dung của bình luận.
- ``createdAt (Date)``: Thời gian tạo bình luận.
- ``updatedAt (Date)``: Thời gian cập nhật bình luận lần cuối.
- Phương thức:
 - ``addComment()``: Thêm bình luận mới.
 - ``deleteComment()``: Xóa bình luận.

5. Like

- Thuộc tính:
 - ``likeId (int)``: Mã định danh duy nhất của lượt thích.
 - ``postId (int)``: Mã bài đăng được thích.
 - ``userId (int)``: Mã người dùng thích bài đăng.
 - ``createdAt (Date)``: Thời gian thực hiện hành động thích.
- Phương thức:
 - ``like()``: Thực hiện hành động thích bài đăng.

• Các dịch vụ

6. AuthService

- Phương thức:
 - ``signUp(user: User)``: Đăng ký tài khoản người dùng mới.
 - ``login(email: String, password: String)``: Đăng nhập vào hệ thống.
 - ``forgotPassword(email: String)``: Quên mật khẩu và yêu cầu đặt lại mật khẩu.

7. ProfileService

- Phương thức:
 - ``updateProfile(profile: Profile)``: Cập nhật thông tin hồ sơ người dùng.

8. PostService

- Phương thức:
 - ``createPost(post: Post)``: Đăng bài viết mới.
 - ``deletePost(postId: int)``: Xóa bài viết.

9. SearchService

- Phương thức:

- ``searchPosts(keyword: String)``: Tìm kiếm bài viết theo từ khóa.
- ``searchUsers(keyword: String)``: Tìm kiếm người dùng theo từ khóa

10. NotificationService

- Phương thức:
- ``sendNotification(user: User, notification: String)``: Gửi thông báo tới người dùng.

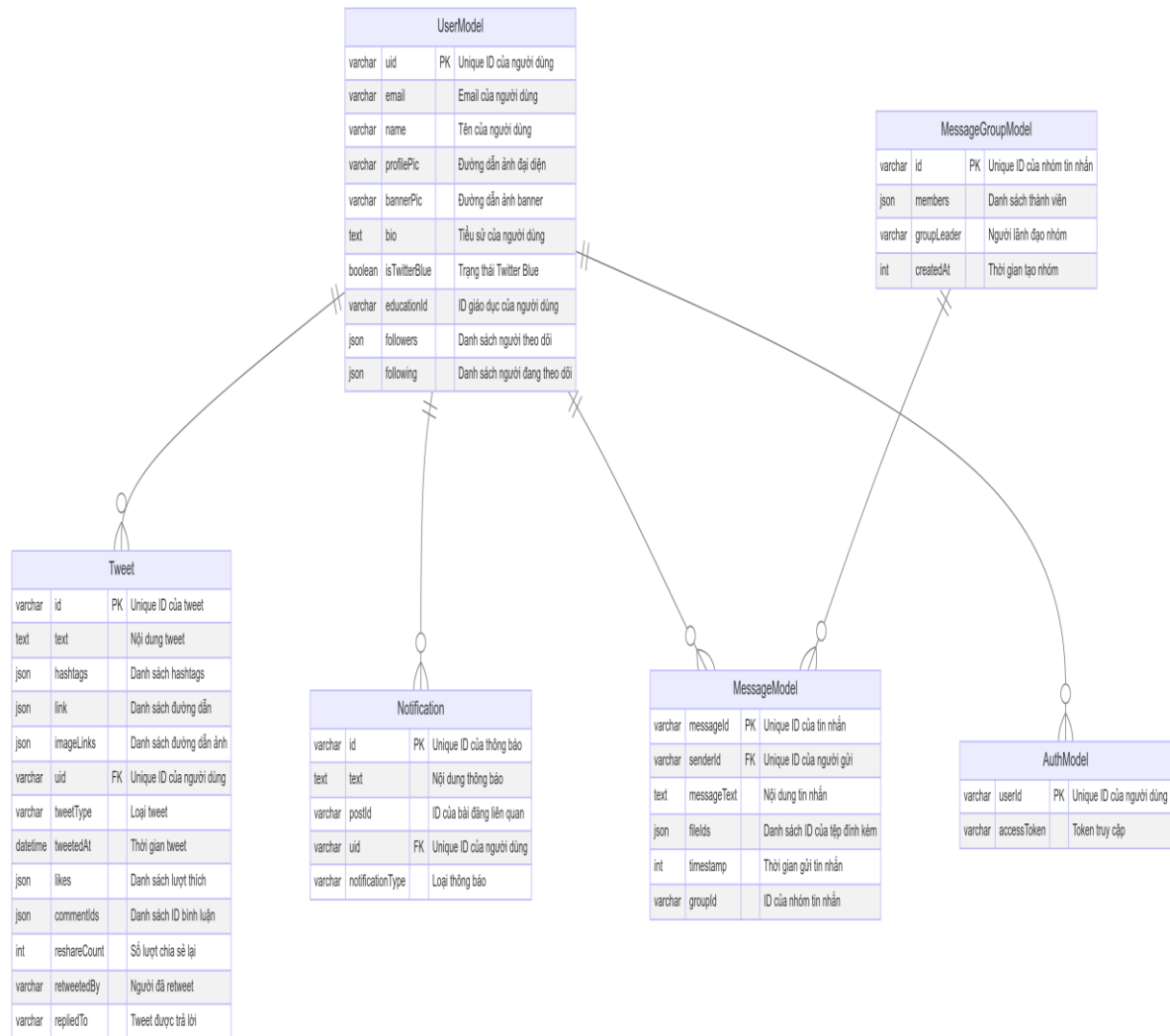
• **Mối quan hệ giữa các lớp**

- User và Profile: Một người dùng có một hồ sơ.
- User và Post: Một người dùng có thể có nhiều bài đăng.
- User và Comment: Một người dùng có thể bình luận trên nhiều bài đăng.
- User và Like: Một người dùng có thể thích nhiều bài đăng.
- Post và Comment: Một bài đăng có thể có nhiều bình luận.
- Post và Like: Một bài đăng có thể có nhiều lượt thích.

• **Sử dụng các dịch vụ**

- AuthService sử dụng lớp ``User``.
- ProfileService sử dụng lớp ``Profile``.
- PostService sử dụng lớp ``Post``.
- SearchService sử dụng lớp ``Post`` và ``User``.
- NotificationService sử dụng lớp ``User``.

2.5. Thiết kế cơ sở dữ liệu



Hình 32: Mô hình thiết kế cơ sở dữ liệu

- **Mô tả thiết kế CSDL:**

1. UserModel

- uid (varchar): Primary Key - ID duy nhất của người dùng.
- email (varchar): Địa chỉ email của người dùng.
- name (varchar): Tên của người dùng.
- profilePic (varchar): Đường dẫn đến ảnh đại diện của người dùng.
- bannerPic (varchar): Đường dẫn đến ảnh banner của người dùng.
- bio (text): Tiểu sử của người dùng.
- isTwitterBlue (boolean): Trạng thái đăng ký dịch vụ Twitter Blue của người dùng.

- educationId (varchar): ID giáo dục của người dùng, có thể liên quan đến các chứng chỉ hoặc khóa học.
- followers (json): Danh sách các ID của người theo dõi (dưới dạng JSON).
- following (json): Danh sách các ID của người đang theo dõi (dưới dạng JSON).

2. Tweet

- id (varchar): Primary Key - ID duy nhất của tweet.
- text (text): Nội dung của tweet.
- hashtags (json): Danh sách các hashtag liên quan đến tweet (dưới dạng JSON).
- link (json): Danh sách các đường dẫn liên quan đến tweet (dưới dạng JSON).
- imageLinks (json): Danh sách các đường dẫn đến ảnh liên quan đến tweet (dưới dạng JSON).
- uid (varchar): Foreign Key - ID của người dùng đã đăng tweet.
- tweetType (varchar): Loại tweet (ví dụ: bình thường, retweet, trả lời).
- tweetedAt (datetime): Thời gian tweet được đăng.
- likes (json): Danh sách các ID của người dùng đã thích tweet (dưới dạng JSON).
- commentIds (json): Danh sách các ID của bình luận trên tweet (dưới dạng JSON).
- reshareCount (int): Số lượt chia sẻ lại của tweet.
- retweetedBy (varchar): ID của người dùng đã retweet tweet.
- repliedTo (varchar): ID của tweet mà tweet hiện tại đang trả lời.

3. Notification

- id (varchar): Primary Key - ID duy nhất của thông báo.
- text (text): Nội dung của thông báo.
- postId (varchar): ID của bài đăng liên quan đến thông báo.
- uid (varchar): Foreign Key - ID của người dùng nhận thông báo.
- notificationType (varchar): Loại thông báo (ví dụ: thông báo bình luận, thông báo thích).

4. MessageModel

- messageId (varchar): Primary Key - ID duy nhất của tin nhắn.
- senderId (varchar): Foreign Key - ID của người gửi tin nhắn.
- messageText (text): Nội dung của tin nhắn.

- fileIds (json): Danh sách các ID của tệp đính kèm (dưới dạng JSON).
- timestamp (int): Thời gian gửi tin nhắn (dưới dạng timestamp Unix).
- groupId (varchar): ID của nhóm tin nhắn (nếu tin nhắn là trong nhóm).

5. AuthModel

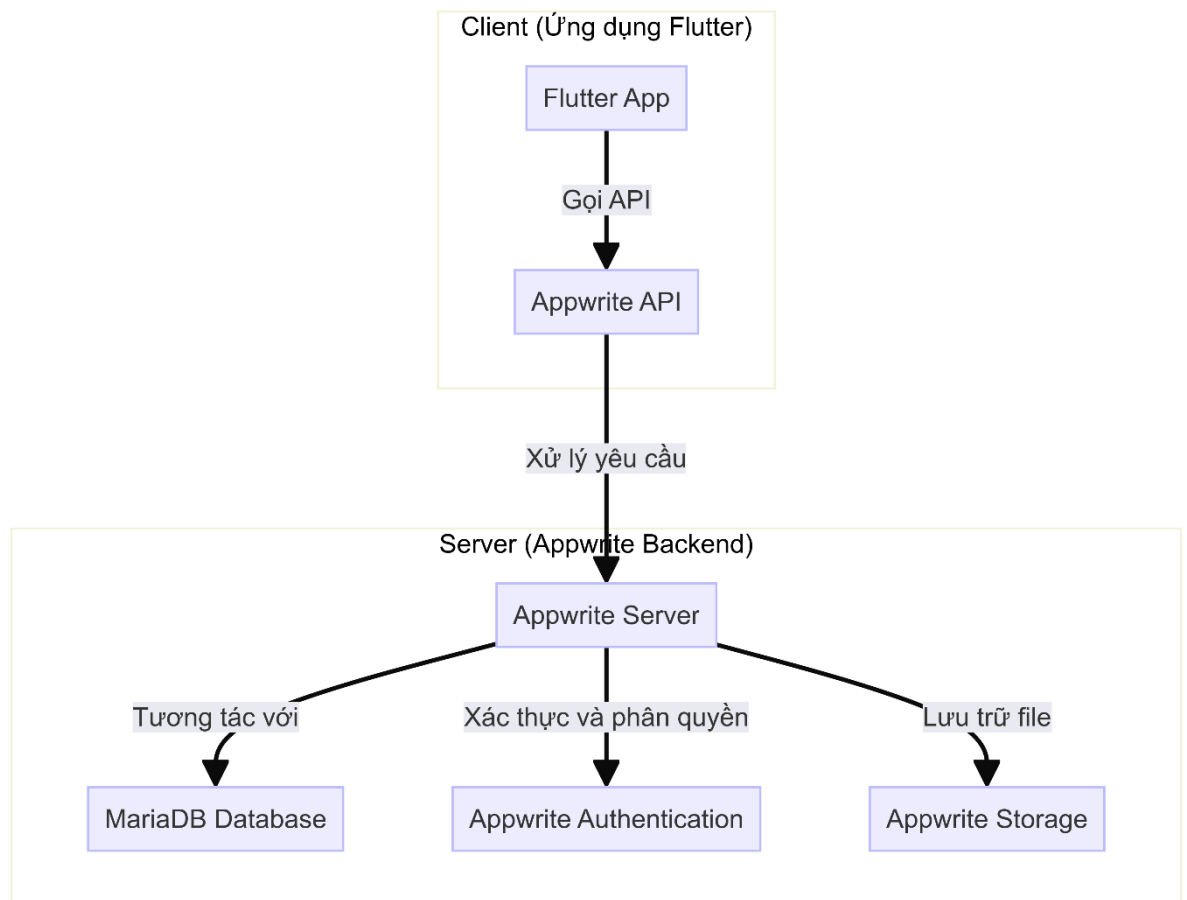
- userId (varchar): Primary Key - ID duy nhất của người dùng.
- accessToken (varchar): Token truy cập cho người dùng.

6. MessageGroupModel

- id (varchar): Primary Key - ID duy nhất của nhóm tin nhắn.
- members (json): Danh sách các ID của các thành viên trong nhóm (dưới dạng JSON).
- groupLeader (varchar): ID của người lãnh đạo nhóm.
- createdAt (int): Thời gian tạo nhóm (dưới dạng timestamp Unix).

3. Thực nghiệm và kết quả

3.1. Mô hình tổng quan



Hình 33: Mô hình hệ thống tổng quan

Mô tả hoạt động:

Client (Ứng dụng Flutter): Giao diện người dùng, Gọi API tới Appwrite backend

Server (Appwrite backend): Cung cấp các API, Xử lý logic nghiệp vụ, Tương tác với cơ sở dữ liệu

Database: Lưu trữ dữ liệu (Appwrite sử dụng MariaDB)

Authentication: Xác thực và phân quyền người dùng (Appwrite cung cấp sẵn)

Storage: Lưu trữ files, ảnh (Appwrite cung cấp sẵn)

Về cơ bản client (ứng dụng Flutter) sẽ giao tiếp với server (Appwrite backend) thông qua các API. Server sẽ xử lý các yêu cầu và tương tác với cơ sở dữ liệu, xác thực người dùng. Appwrite cung cấp sẵn nhiều tính năng như xác thực, lưu trữ cho phép tập trung vào phát triển ứng dụng.

3.2. Các chức năng chính

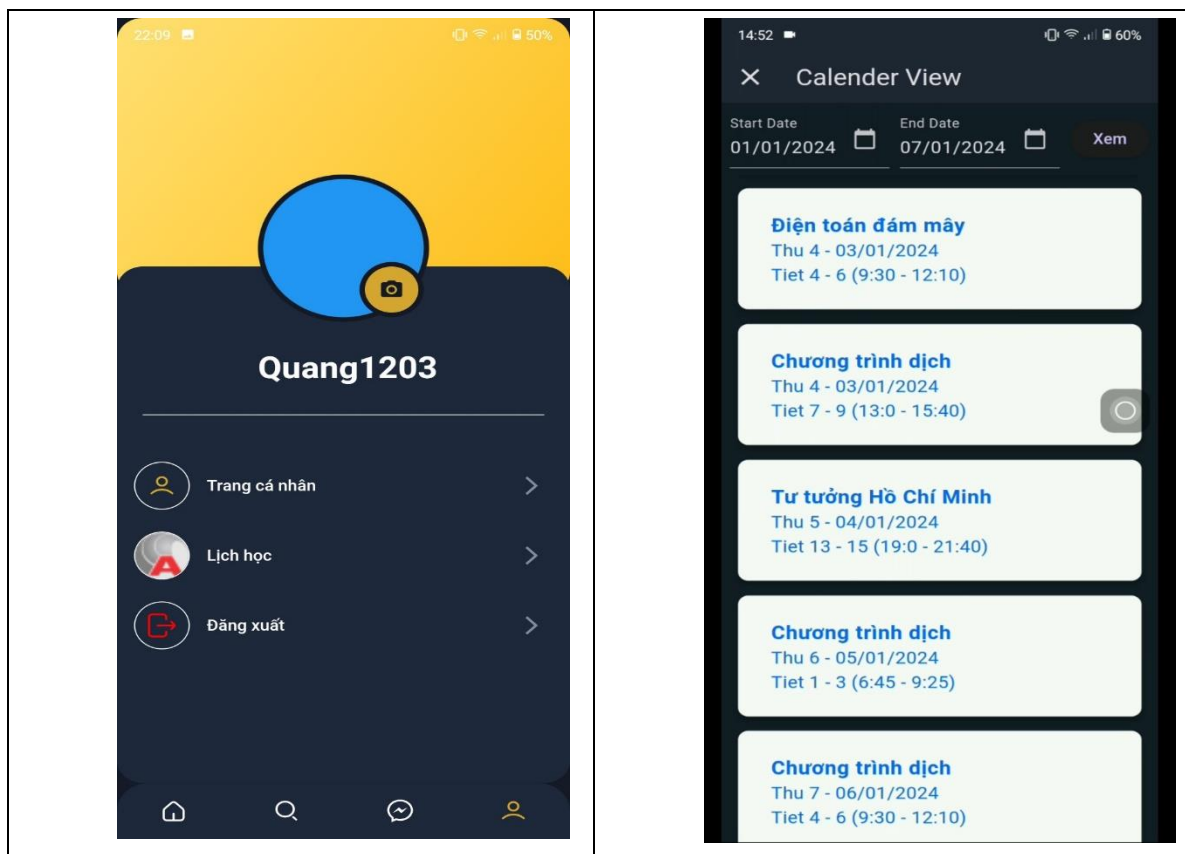
Dưới đây là mô tả chi tiết các chức năng chính của ứng dụng PU Connection:

- **Đăng nhập bằng tài khoản Phenikaa và xem lịch học trên QLDT:**

- Mô tả: Cho phép người dùng đăng nhập vào ứng dụng bằng tài khoản của trường Đại học Phenikaa và đi đến trang lịch học trên QLDT.

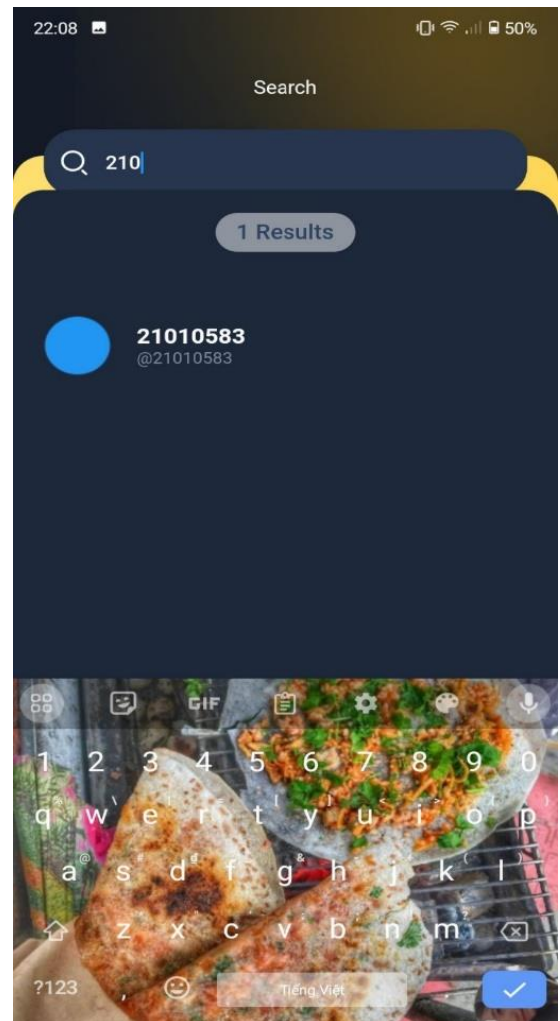
- Quy trình:

1. Người dùng nhập email và mật khẩu của tài khoản Phenikaa.
2. Hệ thống xác thực thông tin đăng nhập với cơ sở dữ liệu của trường.
3. Nếu thông tin đúng, người dùng được chuyển đến trang lịch học trên QLDT.
4. Nếu thông tin sai, hệ thống sẽ thông báo lỗi và yêu cầu người dùng nhập lại.

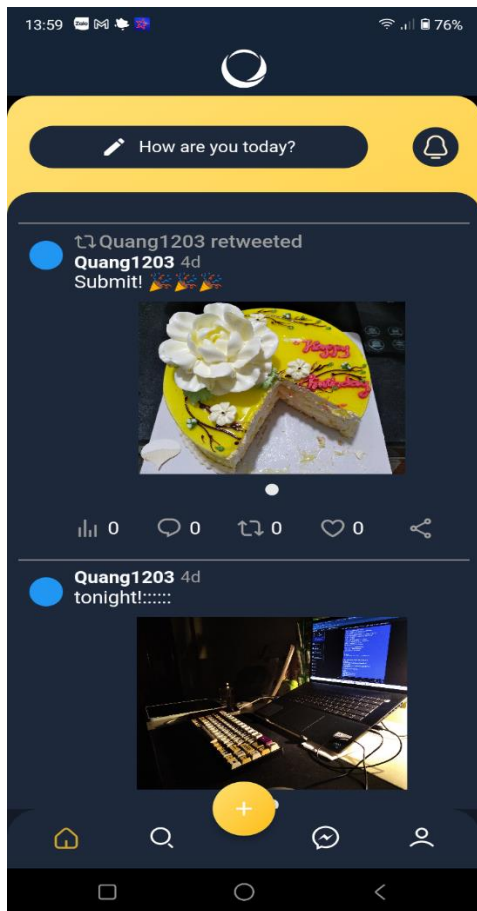


Hình 34: Đăng nhập bằng tài khoản Phenikaa và xem lịch học trên QLDT

- **Tìm kiếm user:**
 - Mô tả: Cho phép người dùng tìm kiếm các người dùng khác trong hệ thống.
 - Quy trình:
 1. Người dùng nhập từ khóa vào thanh tìm kiếm.
 2. Hệ thống thực hiện tìm kiếm trong cơ sở dữ liệu và hiển thị danh sách các người dùng phù hợp với từ khóa.
 3. Người dùng có thể chọn một người dùng từ danh sách để xem chi tiết hồ sơ.



Hình 35: Tìm kiếm user



Hình 36: Giao diện bài viết

- **Đăng status:**

- Mô tả: Cho phép người dùng tạo và đăng các bài viết (status) trên trang cá nhân hoặc cộng đồng.

- Quy trình:

1. Người dùng nhập nội dung bài viết và có thể thêm hình ảnh, video hoặc liên kết.
2. Người dùng nhấn nút đăng.
3. Hệ thống lưu trữ bài viết vào cơ sở dữ liệu và hiển thị trên trang cá nhân của người dùng.

- **Like status:**

- Mô tả: Cho phép người dùng thể hiện sự yêu thích đối với các bài viết của người dùng khác.

- Quy trình:

1. Người dùng nhấn vào nút like trên bài viết mà họ muốn thể hiện sự yêu thích.
2. Hệ thống ghi nhận hành động này và cập nhật số lượng like trên bài viết.
3. Chủ bài viết nhận được thông báo về lượt like mới.

- **Share status:**

- Mô tả: Cho phép người dùng chia sẻ các bài viết của người dùng khác lên trang cá nhân của mình.

- Quy trình:

1. Người dùng nhấn vào nút share trên bài viết mà họ muốn chia sẻ.
2. Hệ thống tạo một bản sao của bài viết và đăng lên trang cá nhân của người dùng.
3. Bài viết gốc và bài viết chia sẻ đều hiển thị thông tin về số lần chia sẻ.

- **Messenger:**

- Mô tả Cho phép người dùng gửi tin nhắn trực tiếp cho nhau trong hệ thống.

- Quy trình:

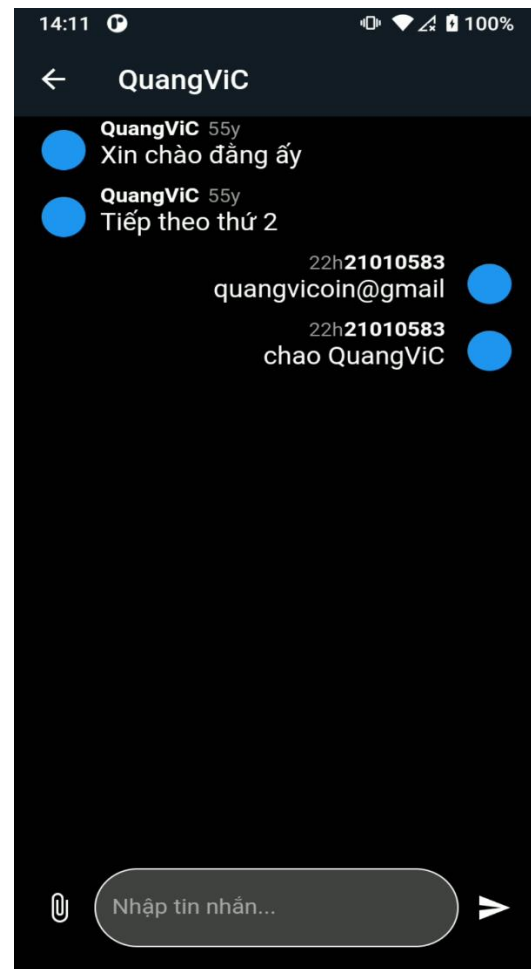
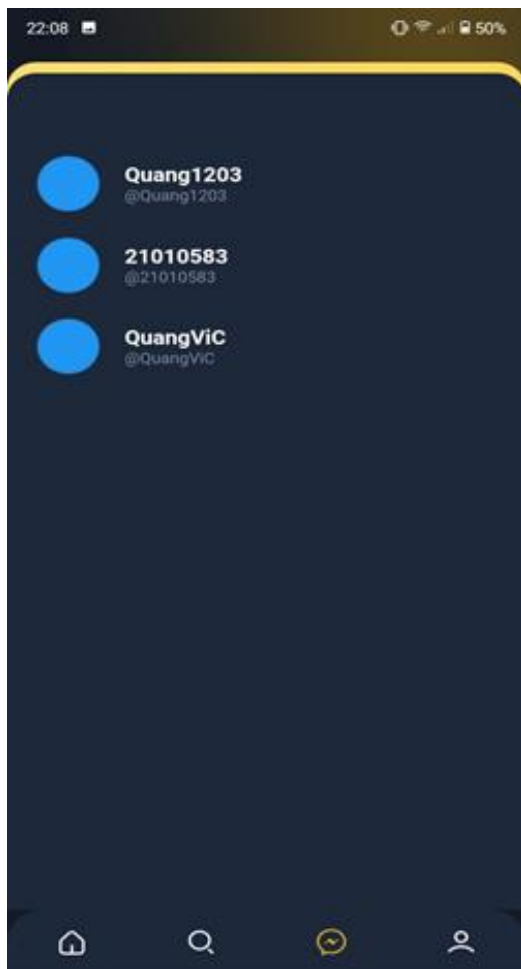
1. Người dùng truy cập vào phần nhắn tin.

2. Người dùng chọn một người dùng khác từ danh sách bạn bè hoặc tìm kiếm người dùng mới để nhắn tin.

3. Người dùng nhập nội dung tin nhắn và nhấn nút gửi.

4. Hệ thống gửi tin nhắn đến người nhận và lưu trữ tin nhắn trong cơ sở dữ liệu.

5. Người nhận nhận được thông báo về tin nhắn mới và có thể đọc và trả lời tin nhắn.



Hình 37: Giao diện messenger

4. Kết luận và hướng phát triển

4.1. Kết luận

Trong quá trình phân tích kỹ thuật và thiết kế ứng dụng PU connection, chúng tôi đã tập trung vào đáp ứng các yêu cầu cụ thể của ứng dụng mạng xã hội dành cho sinh viên tại trường Đại học Phenikaa. Dưới đây, chúng tôi tổng hợp những điểm quan trọng và kết quả của dự án:

- Phân Tích Yêu Cầu:

- Đã thực hiện phân tích chi tiết các yêu cầu chức năng và phi chức năng của ứng dụng.

- Xác định rõ các luồng nghiệp vụ và các trường hợp sử dụng (use-cases) quan trọng.

- Thiết Kế Hệ Thống:

- Thiết kế kiến trúc hệ thống dựa trên các công nghệ hiện đại như Flutter, Appwrite, MariaDB.

- Tạo ra các mô hình thiết kế lớp và cơ sở dữ liệu chi tiết, đảm bảo tính nhất quán và hiệu quả trong việc lưu trữ và xử lý dữ liệu.

- Công Nghệ Sử Dụng:

- Flutter: Đã sử dụng để phát triển giao diện người dùng mượt mà và đẹp mắt cho cả Android và iOS.

- Appwrite: Cung cấp các API backend mạnh mẽ, hỗ trợ xác thực người dùng, lưu trữ files, và nhiều dịch vụ khác.

- MariaDB: Đã được sử dụng làm hệ quản trị cơ sở dữ liệu, cung cấp khả năng lưu trữ dữ liệu quan hệ một cách hiệu quả.

- Các Tính Năng Chính:

- Đăng ký và đăng nhập người dùng.

- Quên mật khẩu và phục hồi tài khoản.

- Đăng bài, tìm kiếm và tương tác với các bài đăng (like, comment, share).

- Quản lý hồ sơ người dùng.

- Gửi và nhận thông báo.

Mục tiêu tạo ra một nền tảng mạng xã hội cho sinh viên với các tính năng chính như đăng nhập, đăng ký, đăng bài, tương tác, và tạo hồ sơ cá nhân. Tất cả những yêu cầu này đã được đáp ứng và tích hợp thành công vào ứng dụng PU connection.

Chúng tôi đã sử dụng framework Flutter để phát triển giao diện di động và sử dụng Appwrite làm nền tảng lưu trữ dữ liệu và xác thực người dùng. Sự kết hợp này đã giúp chúng tôi xây dựng một ứng dụng di động hiệu quả với khả năng quản lý người dùng và dữ liệu mạng xã hội.

Đã triển khai thành công ứng dụng "PhenikaaCampus" trên nền tảng di động và cấu hình cơ sở dữ liệu cho nó. Dự án đã xây dựng một cơ sở dữ liệu với các bảng "Users" và "Tweets" để lưu trữ thông tin người dùng và bài viết.

Trong quá trình dự án, chúng tôi đã học hỏi nhiều về quá trình phân tích thiết kế phần mềm.

Tin rằng ứng dụng PU connection sẽ mang lại giá trị cho cộng đồng sinh viên tại trường Đại học Phenikaa. Và hy vọng rằng bài học từ dự án này sẽ giúp cải thiện và phát triển các dự án phần mềm trong tương lai.

4.2. Hướng phát triển sản phẩm

- **Mở rộng tính năng hiện có**

Tích hợp tính năng tương tác video: Thêm tích hợp video để người dùng có thể chia sẻ và xem video trực tiếp trên ứng dụng.

Hệ thống bình luận cải tiến: Phát triển hệ thống bình luận để hỗ trợ định dạng văn bản, hình ảnh, và emoji.

Thêm hệ thống chatbot AI trả lời các vấn đề liên quan đến học tập.

- **Ứng dụng di động đa nền tảng:**

Hỗ trợ các hệ điều hành khác nhau: Phát triển phiên bản ứng dụng cho các hệ điều hành di động khác nhau như iOS và phiên bản web để mở rộng lượng người dùng.

- **Cải thiện hiệu suất và bảo mật:**

Tối ưu hóa hiệu suất ứng dụng: Tối ưu hóa mã nguồn và dữ liệu để cải thiện hiệu suất và thời gian phản hồi.

Bảo mật dữ liệu: Tăng cường bảo mật dữ liệu và xác thực người dùng để đảm bảo tính an toàn và riêng tư của thông tin.

- **Kết nối với nền tảng bên ngoài:**

Liên kết với nền tảng học tập: Kết nối với các nền tảng học tập và quản lý lịch học để cung cấp thông tin hữu ích cho sinh viên.

4.3. Đóng góp của thành viên nhóm

Thành viên	Công việc
Vi Đăng Quang	- Xác định mục tiêu đề tài - Viết báo cáo - Thiết kế UI - Thiết kế CSDL, backend sever - API cho các module dự án - Xây dựng giao diện bằng flutter - Kết nối frontend và backend - Xây dựng testcase kiểm thử cho phần mềm - Đánh giá tổng quan chức năng, tính thiết thực và hướng phát triển của dự án trong tương lai - Hoàn thiện báo cáo dự án
Phạm Quang Hùng	- Vẽ biểu đồ use-case, activity, sequence - Thêm biểu đồ vào báo cáo dự án

Bảng 11: Bảng phân chia công việc

TÀI LIỆU THAM KHẢO

- Nhung, T. P. (không ngày tháng). *Mô hình quan hệ - thực thể (Entity – Relationship Model)*. Được truy lục từ Thư Viện Học Liệu Mở Việt Nam: <https://voer.edu.vn/m/mo-hinh-quan-he-thuc-the-entity-relationship-model/ff2250a9>
- PlantUML. (không ngày tháng). *Entity Relationship Diagram*. Được truy lục từ PlanUML: <https://plantuml.com/ie-diagram>
- Kteam. (không ngày tháng). *Phân tích thiết kế phần mềm*. Được truy lục từ howKteam: <https://howkteam.vn/course/phan-tich-thiet-ke-phan-mem-33>
- GeeksforGeeks. (không ngày tháng). *Unified Modeling Language (UML) | Sequence Diagrams*. Được truy lục từ geeksforgeeks: <https://www.geeksforgeeks.org/unified-modeling-language-uml-sequence-diagrams/>
- Alan Dennis, B. W. (không ngày tháng). *Systems Analysis and Design: An Object-Oriented Approach with UML*.
<https://dev.to/ooooodiboooo/connecting-your-flutter-application-to-a-backend->

26of

Hà Nội, ngày.... tháng.... năm 2023

Giảng viên hướng dẫn

Sinh viên

Th.S Vũ Quang Dũng

Vi Đăng Quang

Phạm Quang Hùng